



Ca' Foscari
University
of Venice

**Master Degree Programme
in Computer Science and Information technology**

Final Thesis

**Migrating Elixir TypeSpecs to Elixir Types for
LLM inference of Elixir type annotations**

Supervisor

Ch. Prof. Michele Bugliesi

Co-supervisor

Ch. Prof. Giuseppe Castagna

Graduand

Juyoung Kim

Matriculation Number 903635

Academic Year

2025 / 2026

Acknowledgement

I would like to express my deepest gratitude to my supervisor, Professor Michele Bugliesi, for first introducing me to the world of functional programming and type systems, and for supporting this thesis through to completion by offering me a fellowship. His generous and patient mentorship has been invaluable in helping me push through—not only when I was first stepping through the door of lambda calculus, but throughout the entire master’s programme in Venice and the internship in Paris. I am also grateful to Professor Giuseppe Castagna for kindly hosting me at IRIF. His brilliant and rigorously scientific way of reasoning through problems was a great help with the challenges I faced during the formalisation of the translation. I am equally thankful to Dr. Guillaume Duboc and José Valim for always providing me with the right resources and answers—even to my most trivial questions—throughout the whole process.

Declaration on the Use of AI Tools

Generative artificial intelligence was used to support this thesis. The tool was *Claude Code* (Anthropic), based on the *Sonnet 4.6*, *Opus 4.8*, and *Opus 5.0* models, used during 2026. It was employed for three purposes: the revision and copy-editing of prose to improve clarity, coherence, and internal numerical consistency; the drafting of sentences that were subsequently reviewed and reworked by the author; and software-engineering and data-analysis support, namely assistance in debugging the dataset-construction and evaluation pipeline, in computing the reported statistics, and in generating the result and appendix tables from experimental data.

This assistance concerns the chapters on the implementation of the theory and on type prediction with LLMs, together with the abstract, conclusion, contributions, and appendices; it does not concern the formalisation chapter, which was authored independently. No experimental result was generated by the tool: all training runs and evaluations were carried out by the author and recomputed directly from the data and the compiler. Output produced by the tool was checked against the source code and the compiler's behaviour, corrected where it was found to be inaccurate, and recompiled and proof-read; all substantive scientific decisions were taken by the author, who assumes full responsibility for the accuracy, quality, and scientific integrity of the work.

Abstract

This thesis addresses the problem of type inference for the Elixir programming language in two complementary parts. The first part develops a formal translation from Elixir’s TypeSpecs, the language’s long-standing type annotation mechanism, to Elixir Types, Elixir’s new set-theoretic types recently introduced into the language. We provide a formalisation of this translation and a corresponding implementation, which is systematically tested and validated to ensure that the generated types faithfully preserve the semantics of the original TypeSpec annotations. This translation serves both as a contribution in its own right and as the enabling infrastructure for the second, empirical part of the work.

The second part investigates the fine-tuning of large language models (LLMs) for inferring Elixir Types over legacy, unannotated Elixir code. Leveraging the translation developed in the first part, we construct a training dataset from a corpus of open-source Elixir repositories already annotated with TypeSpecs: each TypeSpec is converted into the corresponding Elixir Type, and the resulting (code, type) pairs are retained only when they and their type-check successfully under Elixir’s native typechecker after also pass successfully by Dialyzer with TypeSpecs, thereby guaranteeing that supervision signals are well-typed by construction. Model inference is evaluated not merely by exact match but against Elixir’s notion of type-compatibility—an extension of subtyping to the setting of gradual types—which provides a semantically meaningful measure of correctness. We experiment with two model families, a decoder-only architecture (Qwen2.5-Coder 7B) and an encoder-decoder (CodeT5+ 770M) architecture, and carry out a comparative assessment of their performance across models and across a range of training hyperparameters.

Contents

Introduction	7
Introduction	8
Gradual type system transition from dynamic to static	8
Background	8
TypeSpecs	9
Dialyzer	9
Elixir Types	11
Problem Statements	13
Contributions	14
I Translating TypeSpecs into Elixir Types	15
1 Formalisation	16
1.1 TypeSpecs Syntax Specification	16
1.1.1 TypeSpecs Built-in Types and Syntactic Sugar	18
1.2 Elixir Types Syntax Specification	20
1.2.1 Elixir Type Built-in Types and Syntactic Sugar	21
1.3 Translation from TypeSpecs to Elixir Types	22
1.3.1 Translation of Annotations	22
1.3.2 Translation of Basic Types	24
1.3.3 Translation of Fields	26
1.4 Total Approximating Translation of TypeSpecs into Elixir Types	26
1.4.1 Approximation of Field Types	27
2 Implementation and Analysis	29
2.1 Implementation of theory	29
2.1.1 Translator	29
2.1.2 Migrator	33
2.1.3 Dataset producer	34
2.2 Dataset Construction	34
2.2.1 Overview	34
2.2.2 Dataset Entry Schema	35
2.2.3 Stage 1: Translation Execution	35
2.2.4 Stage 2: Dialyzer Analysis	36
2.2.5 Stage 3: Typechecker Execution	37
2.3 Dataset Analysis	38
2.3.1 Hypothesis	38
2.3.2 Overall Results and Verdict	38
2.3.3 Approximation via <code>dynamic()</code>	39
2.3.4 Dialyzer-Only Warnings: Why Dialyzer Catches What the Typechecker Cannot <i>Yet</i>	40
2.3.5 Typechecker-Only Warnings	42
2.3.6 Entries Flagged by Both Tools	44

2.3.7	Implications for Downstream Use	44
2.3.8	Summary	45
II	Inferring Elixir Types with Large Language Models	46
3	Type Prediction with LLMs	47
3.1	LLM Type Prediction	47
3.1.1	Task Formulation	47
3.1.2	Dataset Preparation	48
3.1.3	Model and Training	49
3.1.4	Evaluation: Set-Theoretic Compatibility	49
3.1.5	Architecture Comparison: A Smaller Encoder–Decoder	50
3.1.6	Qualitative Analysis	51
	Conclusion	52
	Conclusion	53
	Conclusion and Future Work	53
	Limitations and Future Work	53

Introduction

Introduction

Gradual type system transition from dynamic to static

Elixir is a dynamically typed functional programming language that is currently undergoing the integration of a gradual set-theoretic type system Valim (2012). Traditionally, Elixir has relied on its type specification directive, `TypeSpecs`, an Erlang-inspired type annotation system, to document code and support static analysis tools Elixir Core Team (2024c). In particular, `Dialyzer` serves as the de facto standard tool for analysing `TypeSpecs` and detecting discrepancies through success typing Lindahl and Sagonas (2006).

However, `TypeSpecs` is not enforced by the compiler and therefore does not provide strong guarantees of type safety. Instead, it primarily serves as documentation and as optional input to external analysis tools such as `Dialyzer`. While `Dialyzer` is effective in avoiding false positives, it may produce warnings that are difficult to interpret and may lack precise information about the origin or cause of type inconsistencies.

To address these limitations, recent work on Elixir introduces a gradual set-theoretic type system known as Elixir Types Duboc (2024). This system aims to combine the flexibility of dynamic typing with the rigor of static typing. Its benefits can be understood along three main dimensions. First, it improves type safety guarantees by enabling more precise type inference and checking aligned with program behaviour. Second, it supports gradual typing through the inclusion of the `dynamic()` type, allowing parts of the program to remain dynamically checked at runtime. Third, it leverages set-theoretic constructs such as union, intersection, and negation types, enabling more expressive and compositional type definitions.

In this context, an important challenge arises: how to migrate from the existing `TypeSpecs` to the new Elixir Types. To preserve semantics and maximise simplicity in the transition, this paper first investigates the translation of `TypeSpecs` into gradual set-theoretic types, identifying mismatches between the two systems and proposing approximation strategies where direct translation is not possible. Then, we bring this formalisation to implementation; a prototype is developed to evaluate the feasibility and limitations of this approach in practice. After completing the implementation by passing test codes prescribed, we apply fine-tuning technique to train a large language model with translated types and predict Elixir Types on untyped function definitions.

Background

Elixir is a dynamically and strongly typed functional programming language built on top of the Erlang virtual machine (BEAM) Valim (2012); Armstrong (2007). It is designed for building scalable and maintainable applications, particularly in the domain of concurrent and distributed systems.

The dynamic typing in Elixir allows developers to write code without having to specify types explicitly, which can lead to faster development and greater flexibility. Nevertheless, this flexibility can lead to potential runtime errors due to the strongly typed nature, which could have been caught at compile time if a static type system were in place. As the community grows, the yearn for a more robust type system that can provide stronger guarantees about type safety and catch more errors at compile time has become increasingly evident. To meet this demand, the Elixir community has adopted a variety of tools and features to enhance type safety, including `TypeSpecs` and `Dialyzer` for static analysis. Even though these tools have been helpful in improving code quality, they have limitations in terms of expressiveness and the ability to capture complex type relationships. On top of that, the lack of a static type system that could force type checking at compile time has motivated the exploration of more advanced type systems, i.e., Elixir Types.

In this section, we will provide an overview of `TypeSpecs` and `Dialyzer`, and then introduce Elixir Types as a more expressive type system that addresses the limitations of the existing tools.

TypeSpecs

TypeSpecs is a distinct notation for declaring types and specifications based on Erlang with attributes such as `@spec`, `@type`, `@opaque`, and `@typep` Elixir Core Team (2024c); Ericsson AB (2024c). As the name suggests, `@spec` is the one that specifies the types of function parameters and return values:

```
1 @spec id(any()) :: any()  
2 def id(x), do: x
```

This snippet shows a simple form of `@spec` usage where the function `id/1` takes a parameter of type `any()` and returns a value of the same type. In particular, `@spec` can be used with guards that is specified by entailing `when` clause:

```
3 @spec id(my_guard1) :: my_guard2 when my_guard1: any(), my_guard2: any()  
4 def id(x), do: x
```

This specification tells that the function `id/1` takes `my_guard1` and returns `my_guard2` where the both guard variables are `any()`.

As the guard variable types are used to describe a type (mind the name "guard" that it does not actually guard anything, but works as a type variable in a local scope), the rest of the attributes are used in the same manner. The type definition attributes are to define custom types, which we may call user-defined types. The following snippet shows the typing of user-defined types and a usage case in `@spec` annotation:

```
5 @type my_type :: any()  
6 @opaque my_opaque :: my_type  
7 @typep my_typep :: my_opaque  
8  
9 @spec id(my_typep) :: my_typep  
10 def id(x), do: x
```

The difference between these definitions lies in their visibility: `@type` defines a public type that can be used both within the module where it is defined and outside of it; `@opaque` defines an opaque type whose definition is hidden from other modules, meaning that it can be used everywhere but its internal definition is invisible outside of the module; and `@typep` defines a private type that can only be used within the module.

On top of that, type definitions can come with parameters as such:

```
11 @type my_list(some) :: list(some)  
12  
13 @spec id(my_list(any())) :: my_list(any())  
14 def id(x), do: x
```

The usage is the same as the basic user-defined types, but this requires specifying its parameter type in the annotation.

Moreover, one can use parameter's name in a specification:

```
15 @spec id(x :: any()) :: any()  
16 def id(x), do: x
```

This is saying that `id/1` takes a parameter (`x`) which type is (`any()`).

Essentially, all these attributes allow developers to create clear and maintainable code by providing explicit type information, which can be leveraged by tools like Dialyzer for static analysis and error detection.

Dialyzer

Dialyzer is the de facto standard static analysis tool that originally comes from Erlang Lindahl and Sagonas (2006). Its analysis is based on success typing Lindahl and Sagonas (2006), which guarantees the absence of false positives, meaning that if Dialyzer creates a warning, it corresponds to a definite type inconsistency in the code. It is designed to analyse code to detect type errors, unreachable code, and discrepancies with TypeSpecs and inferred types.

Examples of Discrepancy Analysis

Let us consider the following corresponding three examples to illustrate its capabilities:

```
17 @spec add_one(integer()) :: integer()
18 def add_one(x), do: x + "1"
```

In this example, Dialyzer detects a definite type error when the function attempts to add an integer and a string. It infers that the operator `+` expects numeric operands, and therefore reports a type error. Since this operation will always fail at runtime, the warning is guaranteed to correspond to a real issue.

```
19 @spec check(boolean()) :: :ok | :error
20 def check(x) do
21   if x == true do
22     :ok
23   else
24     if x == false do
25       :error
26     else
27       :unknown
28     end
29   end
30 end
```

Dialyzer can identify code branches that can never be executed based on type information.

With this code, the variable `x` is specified to be of type `boolean()`, which can only be `true` or `false`, that concludes the final branch returning `:unknown` unreachable. Dialyzer finds that this branch can never be executed and reports it as dead code.

```
31 @spec always_true() :: true
32 def always_true(), do: :ok
```

In this case, TypeSpecs declares that the function returns the atom `true`, while the implementation returns `:ok`. Dialyzer infers the return type as `:ok`, which contradicts the specification, and reports a type discrepancy.

Such discrepancies indicate that either the specification is incorrect or the implementation does not conform to its intended behaviour, and therefore require attention from the users to resolve the potential bugs in the code.

Examples of limitations

Although Dialyzer is helpful for identifying type issues early in the development process, the conservative approach that avoids false positives comes at the cost of missing some type errors. Especially those that involve complex type relations, or dynamic features of Elixir. To best explain these limitations, we can look at examples where Dialyzer fails to detect type errors:

```
33 @spec maybe_add(integer() | binary()) :: integer()
34 def maybe_add(x) do
35   if is_integer(x) do
36     x + 1
37   else
38     x + 1
39   end
40 end
```

The function `maybe_add/1` is specified to accept either an integer or a binary (string), but it attempts to add 1 to the input regardless of its type. Dialyzer does not report an error here because it cannot conclusively determine that the operation will fail at runtime, in other words, it cannot guarantee that the branch where a binary is passed will *always* be executed, and thus it does not raise a warning. However, this code will indeed raise a runtime error when a binary is passed, since adding an integer to a string is not valid in Elixir.

```
41 @spec get_age(list(%{atom() => binary()})) :: integer()
42 def get_age(person) do
```

```

43  hd(person).age + 1
44  end

```

The function assumes that the head of the list is a map containing the key `:age` with an integer value. There are two types of potential runtime errors: if the map is empty, the `hd/1` function raises an argument error, and if the key is missing, the dot access raises a key error; and even if a member exists the plus operation does not match with the specification. Since Dialyzer cannot guarantee that the list will always contain a map with the required key, it does not report this as a definite error, even though it is a potential source of runtime errors. Likewise the wrong operation is not reported since the value is not guaranteed not to be `integer()` or `float()`

```

45 @spec apply_fun((integer() -> integer()), any()) :: integer()
46 def apply_fun(f, x) do
47   f.(x)
48 end

```

The function `apply_fun/2` expects a function that operates on integers. However, the argument `x` is of type `any()`, which may not be an integer. This may lead to runtime errors if a non-integer value is passed. Due to the generality of the types involved, Dialyzer is not able to detect this issue.

These examples highlight a fundamental limitation of Dialyzer. While its success typing approach guarantees the absence of false positives, it may fail to detect certain classes of errors, particularly those involving unions of types, partial data structures, and higher-order functions. As a result, the inferred types may be overly general, and potential runtime errors may remain undetected.

This limitation motivates us to explore more expressive type systems: set-theoretic types, which aim to capture more precise relationships between values and enable stronger guarantees about program behaviour.

Elixir Types

Elixir Types is a gradual type system based on the semantic subtyping Frisch et al. (2008) that utilises set-theoretic types Duboc (2024) to provide a more precise and expressive way to describe the types of functions and data structures. It is designed to be sound, meaning that if a program type checks successfully, it is guaranteed to be free of type errors at runtime, that is, no false negative opposed to Dialyzer.

The type system requires two main features to type check the code: explicit type annotations and local type inference. The latter is achieved through a phic type system with a more sophisticated local type inference mechanism. Which means that while the type system looks for explicit type annotations on functions, it can also deduce the types of function bodies and their interactions based on the code structure and the operations performed on the data. This can capture complex type relationships and constraints that Dialyzer may miss.

Gradual type system

What gradual type system means is that the type system allows for both static and dynamic typing Siek and Taha (2006), enabling users to choose the degree of static type checking. For this reason, Elixir Types introduces a special type called `dynamic()`, which essentially puts the type-checker in a "dynamic mode": for any expression represented by or intersected with type `dynamic()`, the type system does not perform static checks and let a type to be determined at runtime. By default, the new type system replaces absence of type annotations with `dynamic()`, which gives leniency to users who are not ready to fully commit to static typing. Another power of this approach is that its presence in the type system guarantees that when an expression is of a given type, it will either diverge, return a value of the given type, or fail on a runtime type check verification.

In the perspective of transitioning from untyped to typed code, this allows smooth type adoption for existing users: as users are already familiar with dynamic features of Elixir, they can incrementally add type systems without having to rewrite their entire well-working codebase. Which is a significant advantage, for that it lowers the barrier to entry for accepting a more robust type system and encourages gradual improvement of code quality over time without causing unwanted errors by forcing the type system.

Moreover, it enables the use of set-theoretic types, which can express unions, intersections, and negations of types, providing richness in describing the behaviour of functions and data structures. This allows developers to write more precise type annotations and catch a wider range of type errors at compile time, ultimately leading to safer and more maintainable code.

Set-theoretic types

Set-theoretic types Frisch et al. (2008); Castagna et al. (2014) provide a powerful way to express complex type relationships using operations like union, intersection, and negation. These types satisfy the commutativity and distributivity properties of their counter-part in the set-theoretic operations. We illustrate these types as follows:

Union types can represent a value that can be of multiple types. For example, an `integer()` type is a set of union types of all the integer types hence a value annotated with `integer()` can be an integer, and `number()` is defined as `integer() or float()`, which represents a set of all the integer types and float types. The function types can also be defined as union types, for instance, the type of a function that takes an integer and returns an integer can be represented as `(integer() -> integer())`, and the type of a function that takes either an integer or a float and returns correspondingly either an integer or a float can be represented as `((integer() -> integer()) or (float() -> float()))`.

Intersection types can represent a value that must satisfy multiple type constraints simultaneously. For example, the type of a value that is both an integer and a float can be represented as `integer() and float()`, which results in the empty set, `none()`. Although the representation of intersection types for value types is intuitive, that of intersection types for function types is more nuanced. Instead of intersecting each inputs and outputs, it expresses a function that must take both terms of an intersection type. Hence, an intersection type of functions `(integer() -> integer())` and `(float() -> float())` is not `none() -> none()` but it represents the type of a function that must contain both `(integer() -> integer())` and `(float() -> float())` as subtypes.

Negation types can represent the absence of certain types, that is, the complement of a type. For example, the type of a value that is not an integer can be typed with a set of all the types that are not integers, denoted as `not integer()`. Notice that it is another way of representing `term()` and `not integer()` (n.b., we use `term()` instead of `any()` here because the Elixir Types uses `term()`), which is more intuitive to understand as the set of all the terms that are not integers. Therefore, in consecutive of the previous example, the type of value `number()` and `not integer()` can be represented as `float()`, which is the set of all the numbers that are not integers, and thus only contains floats. We can also use negation types to express function types, however, it is trivial to show the use of negated function types, because we do not have to describe what is not necessary for a function type to be (i.e., `(integer() -> integer())` is the same as `(integer() -> integer())` and `not float() -> float()`, since the function type is already specific enough to exclude any other function types).

This expressiveness allows developers to capture more nuanced type relationships and constraints that may arise in practice, leading to better type safety and more accurate type checking.

Tackling limitations of Dialyzer on TypeSpecs

We have seen that Dialyzer can miss certain type errors due to its conservative approach to avoid false positives. Elixir Types, with its more expressive type system, can capture these errors by providing more precise type annotations and leveraging set-theoretic types to express complex type relationships. Let us revisit the examples of limitations in Dialyzer and see how Elixir Types can address them:

```
49 $ (integer() or binary()) -> integer()
50 def add(x) do
51   if is_integer(x) do
52     x + 1
53   else
54     x + 1
55   end
56 end
```

By the annotation, the type system knows that the function accepts either an integer or a binary, but it infers that the second branch of the function is not valid for a binary input. The type system can infer this by recognising that the operation `+` is only valid for integers and floats, and thus it can determine that the second branch will always fail when a binary is passed, producing an incompatibility error.

```

57 $ list(%{atom() => if_set(binary())}) -> integer()
58 def get_age(person) do
59   hd(person).age + 1
60 end

```

What the type system sees first is that the list type can be either empty or non-empty. Since the function does not handle the empty list case, the type system raises an error. To observe potential errors lying behind, we can first rewrite the annotation as: `$ list(%atom() => if_set(binary())) and not empty_list() -> integer()`. With the negation operator, we can express that the input list must be non-empty, which allows the type system to infer that the `hd/1` function will always succeed, and thus it can proceed to check the rest of the function body. Then, it can infer that if the dot access is valid for a map containing the key `:age`, which is not guaranteed by the annotation because the specified key type is `atom()`. Thus, by finally rewriting it to `$ list(%age: binary()) and not empty_list() -> integer()`, the type system can determine that the `+` operation will fail when the value is a binary, producing an incompatibility error.

```

61 $ (integer() -> integer()), term() -> integer()
62 def apply_fun(f, x) do
63   f.(x)
64 end

```

With the set operations, the type system can deduce that `term()` breaks down to `integer()` or `not integer()`. Therefore, it can capture that the function application will fail when `x` is not an integer, producing an incompatibility error.

Problem Statements

The first problem to be addressed by this paper is the friction in translation that is caused by the syntactical gap between TypeSpecs and Elixir Types. As Elixir is built on top of Erlang, their semantics match one-to-one, which means that Erlang can achieve everything Elixir can do, and vice versa. However, the major difference of two languages comes from their syntaxes; Elixir is designed to be simpler than Erlang, that is, to remove boilerplates and to reduce noise from Erlang in order to maximise usability while keeping the advantages of high availability. Accordingly, the principle difference between TypeSpecs and Elixir Types originates from the syntactical difference of the two languages. Since the initial intention of TypeSpecs is to compensate the lack of a static type system in Erlang, it reflects granularity of Erlang syntax. That is, TypeSpecs allows specifying literal types such as atoms, binaries, and digits, whereas, Elixir Types pursues simplicity and does not support all of the literal types but atoms.

This problem is crucial for developers in the process of moving forward to Elixir Types. The new Elixir type system is meant to perform more precise type check, providing useful information. However, this advantage could fade out, for example, with abuse of the union type or the gradual type (i.e., `or` or `dynamic()`, respectively) everywhere to mute type errors. This misuse of types could arise easily when transition is attempted without any guidelines. Hence, the main purpose of this paper is to provide safe and clear translation and eventually migration guidance; since the more precise type system could generate errors unseen before on codebases that run without problems.

The translation is the transition from a subtype (TypeSpecs) to a supertype (Elixir Types) since Elixir Types does not support types that are not frequently used for design simplicity while it can cover all the existing TypeSpecs. We will explore deeper into this in the formalisation chapter by considering each type to discover types that can be directly translated and types that need to be approximated.

The second problem is that the use of TypeSpecs is optional. A safe translation on an annotation can be helpful but it only works when there is a specification while most of Elixir definitions do not have TypeSpecs annotations. This problem is already recognized in Duboc's paper which is suggested to make types gradual with inferred types. Despite its safeness, this may slow down the steps in the whole transition to the ultimate static type system and may allow most of users to settle down on gradual types, which trivially circumvents the compile time type check.

Inferring types with LLMs using fine-tuning has already been studied. We make a reference to an existing paper and create an adaptation suits for Elixir's set-theoretic types in order to train, generate, and evaluate types.

Contributions

Elixir is currently taking a significant step towards integrating a type checker into its compiler. Although the transition is gradual, both in its adoption and in the type system itself, the ultimate goal is to establish a static type system in Elixir. To support this process, we make the following contributions in this paper:

- *Formalisation of TypeSpecs and Elixir Types.* We formalise the existing TypeSpecs based on the TypeSpecs reference and the behaviour of Dialyzer. The formalisation of Elixir Types is drawn from Elixir Type System by Giuseppe Castagna and Guillaume Duboc Duboc (2024).
- *Translation from TypeSpecs to Elixir Types.* We suggest the guideline to safely interpret Formalised TypeSpecs to Elixir Types.
- *Approximating translation of undefined types.* We identify types that are unable to directly translate and introduce conservative techniques to approximate the types.
- *Prototype implementation of the proposed translation.* We apply the developed theory to real world data (public open-source repositories¹) to produce a dataset that contains function definitions, original TypeSpecs, and translated types.²
- *Agreement study: Dialyzer versus the Elixir typechecker.* We measure the produced dataset by means of examining the validity and the quality of translation.
- *LLM type prediction with translated Elixir Types.* We fine-tune a model (Qwen2.5+ 70b) based on the paper by H.B.Mengesha Mengesha (2026) with a filtered dataset that is the entries passed by both Dialyzer and the compiler.³ The evaluation adopts a set-theoretic method by using `comparable?/2` from `Module.Types.Descr` developed by Duboc Guillaume.

¹The 175 source repositories, with the exact commits used during construction, are listed in the appendix (Table 3.3).

²The translation prototype and dataset-construction pipeline are available at https://github.com/KyleJYKim/type_migrator.

³The fine-tuning pipeline is available at https://github.com/KyleJYKim/type_migrator_with_llm.

Part I

Translating TypeSpecs into Elixir Types

Chapter 1

Formalisation

This chapter provides rigorous type syntax formalisation both TypeSpecs and Elixir Types in order to produce theoretical translation equations. Our formalisation starts with TypeSpecs syntax from existing TypeSpecs reference Elixir Core Team (2024c); Ericsson AB (2024c), which lists not only the basic types but also the built-in types and syntactic sugars. The syntax specification of Elixir Types is then provided based on the Featherweight Elixir Duboc (2024). Finally, we introduce the translation equations and discriminate the types that are not able to be directly translated, which leads us to suggest approximation of translation.

1.1 TypeSpecs Syntax Specification

Annotations	$A ::= \overline{@spec\ E}$
Expressions	$E ::= x(\overline{T}) :: T \mid x(\overline{T}) :: T \text{ when } \overline{\alpha} : \overline{T}$
Types	$T ::= \alpha \mid T \mid T$ <code>any()</code> <code>none()</code> <code>atom()</code> <code>:k</code> <code>pid()</code> <code>port()</code> <code>reference()</code> <code>float()</code> <code>integer()</code> <code>neg_integer()</code> <code>non_neg_integer()</code> <code>pos_integer()</code> <code>tuple()</code> <code>{\overline{T}}</code> <code>%{\overline{F}}</code> <code>[]</code> <code>nonempty_maybe_improper_list(T, T)</code> <code>«_::n»</code> <code>«_::*n»</code> <code>«_::n, _::*n»</code> <code>(\overline{T} -> T)</code> <code>(... -> T)</code> <code>n</code> <code>n..n'</code>
Fields	$F ::= \text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names.)

Figure 1.1: TypeSpecs

The syntax specification of TypeSpecs is provided in Figure 1.1 Elixir Core Team (2024c); Ericsson AB (2024c).

An annotation consists of directive `@spec` followed by expression, E . Since an annotation can be empty, one, or multi-clause, we describe it as a sequence by putting overline.

An expression can be entailed by type variables with `when` keyword. Unlike the sequence in an expression, the type variable sequence does not mean that it can be empty because once signified with `when` the clause must contain at least one type variable (Note: The non-empty sequence with overline only applies to type variables).

The types addressed in this specification are the most basic and atomic types of TypeSpec annotations, and they are not defined in terms of other types:

- α represents a type variable, which can be instantiated to any type.
- $T | T$ represents the union of two types, which includes all values that belong to either type.
- `any()` is the top type that is a set (union) of all the types.
- `none()` is the bottom type that is an empty set.
- `atom()` and `:k` represent the type of all atoms and the singleton type of a specific atom, respectively.
- `pid()`, `port()`, and `reference()` represent the types of process identifiers, ports, and references, respectively.
- `float()` and `integer()` represent the type of all decimal point numbers and all digit numbers, respectively.
- `neg_integer()`, `non_neg_integer()`, and `pos_integer()` represent integers with specific interval ranges: `neg_integer()` from -1 to negative infinity, `non_neg_integer()` from 0 to positive infinity, and `pos_integer()` from 1 to positive infinity.
- `tuple()` and $\{\bar{T}\}$ represent the type of all tuples and a tuple type with a sequence of element types, respectively. Although `tuple()` is a supertype of any type represented by $\{\bar{T}\}$, it is necessary to contain both in the syntax specification because `tuple()` is an open tuple type that cannot be described by literal tuple types, e.g., `tuple()` can represent `tuple()` itself but $\{\bar{T}\}$ cannot.
- $\% \{\bar{F}\}$ represent a single map type with a sequence of field types. Notice that we don't include `map()` in the syntax specification because TypeSpecs only has closed map types, which means `map()` can be represented by unions of all the possible field types.
- `[]` and `nonempty_maybe_improper_list(T, T)` represent the type of empty list and non-empty list, respectively. The *maybe improper* indicates that the termination type of a list can be either another list or any other type, which is the reason why the second argument of `nonempty_maybe_improper_list` is not fixed to a list type.
- `«»`, `«_::n»`, `«_::_*n»`, and `«_::n, _::_*n»` represent the types of bitstrings in literal form. The first one is an empty bitstring type, the second one is a non-empty bitstring type with a fixed size n , the third one is a non-empty bitstring type with a size that is a multiple of unit n , and the last one is a non-empty bitstring type that starts with a bitstring of a size n followed by another bitstring.
- $(\bar{T} \rightarrow T)$ and $(\dots \rightarrow T)$ represent the types of function in literal form. The first one is a function type with a fixed arity that takes a sequence of argument types and returns a return type, while the second one is a function type with an arbitrary arity that takes any number of arguments and returns a return type.
- n and $n..n'$ represent the types of integers in literal form. The first one is a singleton type that represents a specific integer n , while the second one is an interval type that represents all integers from n to n' .

A field type can be either a required or optional; intuitively, required field types mandate values of the types to be present in a map, while optional field types allow them to be absent. Notwithstanding the fact that we can syntactically specify required or optional for any key types, the nature of a field is not determined by either keyword. In TypeSpecs with Dialyzer, mandatory types are literal types and all other types are considered optional. Moreover, prescribing a key type is not forced and this causes confusion in field type comprehension. We will discuss more about field types in translation and approximation sections, but for now it is sufficient to say that the field types are necessary to be in the syntax specification because they are the building blocks of map types, which are one of the most commonly used types in TypeSpecs.

1.1.1 TypeSpecs Built-in Types and Syntactic Sugar

TypeSpecs provides a variety of built-in types and syntactic sugars that are not explicitly defined. We include tables for the built-in types and syntactic sugars in Table 1.1 and Table 1.2 for the completeness and smoothness in parsing and translation:

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()

Built-in types	Defined as
node()	atom()
number()	integer() float()
map()	%{optional(any()) => any()}
struct()	%{:__struct__ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

Table 1.1: TypeSpecs Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k: T	:k => T
%k{F̄}	%{:__struct__ => :k, F̄}

Table 1.2: TypeSpecs Syntactic Sugars

The grammar in Figure 1.1 intentionally contains only the core TypeSpecs constructs. However, TypeSpec programs usually use a larger surface language consisting of built-in type aliases and syntactic sugars. These forms do not add new semantic categories; instead, they are expanded into the core syntax before translation. We therefore treat the tables in Table 1.1 and Table 1.2 as a normalisation layer between concrete TypeSpecs syntax and the formal translation rules. Built-in types are mostly derived names for common combinations of primitive types. For instance, `term()` is equivalent to `any()`, `no_return()` is equivalent to `none()`, and `number()` abbreviates `integer() | float()`. Other built-ins describe common refinements, such as `arity()`, `byte()`, and `char()`, or common recursive data structures, such as `iolist()` and `iodata()`. The list-related built-ins are particularly important because TypeSpecs distinguishes empty lists, non-empty lists, proper lists, and maybe-improper lists. Syntactic sugars are different from built-in types: they are alternative concrete notations for existing type expressions. For example, `true`, `false`, and `nil` are parsed as the singleton atom types `:true`, `:false`, and `:nil`. Similarly, `[T]` abbreviates `list(T)`, while `[T, ...]` abbreviates `nonempty_list(T)`. For maps, the field notation `k: T` expands to a singleton atom key, and `struct` notation expands to a map containing the distinguished `__struct__` field. This separation keeps the translation relation small: the formal rules only need to translate the core syntax, while every built-in type or syntactic sugar is first expanded according to the tables.

1.2 Elixir Types Syntax Specification

Annotations	$A ::= \$S \mid \$S \text{ when } \overline{\alpha} : \overline{T}$	
Signatures	$S ::= \overline{T} \rightarrow T \mid S \text{ and } S$	
Types	$T ::= \alpha$ $\mid K \mid \text{term}() \mid \text{none}() \mid \text{dynamic}()$ $\mid T \text{ or } T \mid T \text{ and } T \mid \text{not } T$ $\mid n..n \mid \text{infty}..n \mid n..\text{infty}$ $\mid \{\overline{T}\} \mid \{\overline{T}, \dots\} \mid \% \{\overline{F}\} \mid \% \{\dots, \overline{F}\} \mid (\overline{T} \rightarrow T)$ $\mid \text{empty_list}() \mid \text{non_empty_list}(T, T)$	type variables basic types type connectives integers (internal representation) type constructors
Key types	$K ::= \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{bitstring}() \mid \text{binary}()$ $\mid \text{tuple}() \mid \text{open_map}() \mid \text{fun}()$ $\mid \text{list}()$	
Fields	$F ::= :k \Rightarrow T \mid K \Rightarrow \text{if_set}(T)$	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\{\overline{T}, \dots\}$, $\% \{\dots, \overline{F}\}$, and $\overline{\alpha} : \overline{T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 1.2: Elixir Types Syntax

The syntax specification of Elixir Types is provided in Figure 1.2 and is derived from Featherweight Elixir syntax by Duboc (2024).

This specification focuses on comparison with TypeSpecs for the translation defined in the following sections. An annotation consists of the directive $\$$ followed by a signature, S . Similarly to TypeSpecs, an annotation can be entailed by type variables with the `when` keyword. However, Elixir Types does not use multi-clause annotations in the same way as TypeSpecs. Instead, multiple signatures can be combined inside a single annotation with the set-theoretic connective `and`.

The main difference from TypeSpecs is that the new Elixir type system is explicitly set-theoretic. Each type denotes a set of runtime values, and the type connectives are interpreted as set operations. The type `term()` denotes the set of all values, `none()` denotes the empty set, `or` denotes union, `and` denotes intersection, and `not` denotes complement. For example, `integer()` or `binary()` denotes the values that are either integers or binaries, while `term()` and `not nil` denotes all values except `nil`. In addition, Elixir Types contain the gradual type `dynamic()`. Unlike `term()`, which statically denotes all values, `dynamic()` marks parts of a type whose precision is intentionally unknown. This distinction is important for gradual checking, but in the translation from TypeSpecs it is usually preferable to generate precise types when the source annotation provides enough information.

The types addressed in this specification are the core forms used by Elixir Type annotations:

- α represents a type variable, which can be constrained in a `when` clause.
- `term()`, `none()`, and `dynamic()` represent the top type, the bottom type, and the gradual type, respectively.

- `or`, `and`, and `not` represent union, intersection, and negation. These connectives make it possible to express precise combinations of types without introducing new ad-hoc type constructors.
- `:k` ranges over the key types that can appear as map keys. These include atoms, singleton atoms, process identifiers, ports, references, numbers, bitstrings, binaries, tuples, open maps, functions, and lists.
- `n..n`, `infty..n`, and `n..infty` represent integer intervals. These are treated as an internal representation rather than as ordinary user-facing syntax.
- `{ $\bar{T}$ }` and `{ $\bar{T}$ , ... }` represent closed and open tuple types, respectively. A closed tuple fixes the number of elements, whereas an open tuple allows additional elements after the listed prefix.
- `%{ $\bar{F}$ }` and `%{ ...,  $\bar{F}$  }` represent closed and open map types, respectively. A closed map describes only the listed fields, whereas an open map may contain fields not explicitly listed.
- `empty_list()` and `non_empty_list(T, T)` represent the empty list and non-empty list types. The second argument of `non_empty_list` represents the tail type, which allows the syntax to describe both proper and improper lists.
- `( $\bar{T}$  -> T)` represents a function type with a fixed sequence of argument types and a return type.

Field types in Elixir Types differ from TypeSpecs field types. A singleton atom field `:k=>T` describes a mandatory field whose key must be present. For every other key type, the field has the form `K=>if_set(T)`, meaning that values associated with keys in `K` must have type `T` whenever such fields are present. This distinction makes optionality explicit in the type language and avoids the ambiguity of TypeSpecs fields, where `required` and `optional` do not fully determine the behaviour observed by Dialyzer.

1.2.1 Elixir Type Built-in Types and Syntactic Sugar

Built-in types	Defined as
<code>boolean()</code>	<code>:true</code> or <code>:false</code>
<code>number()</code>	<code>integer()</code> or <code>float()</code>
<code>non_empty_list(T)</code>	<code>non_empty_list(T, empty_list())</code>
<code>list(T, T')</code>	<code>empty_list()</code> or <code>non_empty_list(T, T')</code>
<code>list(T)</code>	<code>empty_list()</code> or <code>non_empty_list(T, empty_list())</code>
<code>list()</code>	<code>empty_list()</code> or <code>non_empty_list(term(), empty_list())</code>

Table 1.3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
<code>{...}</code>	<code>tuple()</code>
<code>%{...}</code>	<code>open_map()</code>
<code>k: T</code>	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 1.4: Elixir Type Syntactic Sugars

As with TypeSpecs, the grammar in Figure 1.2 contains only the core forms of the language. The built-in types and syntactic sugars in Table 1.3 and Table 1.4 are therefore treated as derived forms. The set-theoretic connectives make many derived types straightforward: for instance, `boolean()` expands to the union `:true or :false`, and `number()` expands to `integer()` or `float()`. List aliases are also definable from `empty_list()` and `non_empty_list(T, T')`, which are the primitive list forms in the grammar.

The syntactic sugars mostly provide compact notation for common open structures and map fields. For example, `{...}` denotes the open tuple type `tuple()`, and `%{...}` denotes the open map type `open_map()`. For map fields, `not_set()` is a shorthand for `if_set(none())`, and non-singleton key fields are implicitly optional through `if_set`. Consequently, the translation can target the core Elixir Types syntax after desugaring, while still preserving the set-theoretic interpretation of the generated type.

The translation rules defined in the next section therefore assume all TypeSpec annotations have been fully desugared, i.e., expanded; the implementation applies this normalisation as a parsing pass before applying the rules.

1.3 Translation from TypeSpecs to Elixir Types

In this section, we translate TypeSpecs walking through syntaxes introduced in the previous specifications. The previous sections show that TypeSpecs and Elixir Types describe many of the same runtime values. Both systems provide types for atoms, numbers, tuples, maps, lists, bitstrings, and functions. They also share the basic structure of function annotations: an input type is related to an output type, and type variables can be introduced with a `when` clause. This common ground makes a structural translation possible. For example, a TypeSpec union can often be translated to an Elixir Type union, and a tuple type can often be translated component-wise.

However, the translation is not merely a change of notation. The two systems disagree in several places that matter for precision. TypeSpecs includes many literal forms, such as integer singletons, integer intervals, bitstring size literals, and function literals, while Elixir Types exposes a smaller set of user-facing type constructors. Conversely, the new Elixir type system is explicitly set-theoretic: it provides union, intersection, and negation as first-class type connectives, and it distinguishes precise types from the gradual type `dynamic()`. Moreover, map fields have different forms in the two systems as the set-theoretical type is more precise and unambiguous. TypeSpecs allows `required` and `optional` field declarations for arbitrary key types, but Dialyzer treats mandatory fields reliably only for literal keys. Elixir Types instead makes this distinction explicit through mandatory singleton fields and `if_set` fields.

These disagreements determine the shape of the translation. Where the source construct has an exact counterpart, the translation is direct. Where no exact counterpart exists, the translation relation is intentionally partial in this section, leaving approximation to the following section. This separation lets us state the precise fragment first and then extend it conservatively when total translation is required.

To help formalising translation process, we define translation function $\llbracket \cdot \rrbracket$, which maps TypeSpecs to Elixir Types, formally:

$$\llbracket \cdot \rrbracket: \text{TypeSpecs} \rightarrow \text{Elixir Types}$$

Figure 1.3: Semantic of Translation

We enclose TypeSpecs with $\llbracket \cdot \rrbracket$, and let the translated types be out of them.

1.3.1 Translation of Annotations

The translation of annotations is defined inductively over the structure of TypeSpec annotations. A TypeSpec annotation is a sequence of one or more `@spec` clauses for the functions with the same name and arity (Note that Dialyzer can analyse TypeSpec annotations with arity less than their function arity only when the unspecified types correspond to parameters with default values). Each clause carries a function expression that may be further guarded by type variable constraints introduced with a `when` keyword. The translation maps this sequence to a single Elixir Type annotation, prefixed by `$`, whose signature is a conjunction of translated function types.

Figure 1.4 establishes the outermost annotation layer. A single `@spec E` directive is translated to $\$ \llbracket E \rrbracket$, where the $\$$ prefix marks the result as an Elixir Type annotation substituting `@spec E` directive and $\llbracket \cdot \rrbracket$ translates the expression, E . When the annotation contains multiple `@spec` clauses, the second rule applies recursively: the already-accumulated prefix $\overline{\text{@spec } E}$ is combined with the next clause `@spec E'` by delegating to the expression-level translation $\llbracket \overline{E}, E' \rrbracket$. The overline notation $\overline{E}$ denotes a non-empty sequence of clauses; the translation therefore builds the target signature incrementally, one clause at a time.

$$\llbracket \text{@spec } E \rrbracket = \$ \llbracket E \rrbracket$$

$$\llbracket \overline{\text{@spec } E}, \text{@spec } E' \rrbracket = \$ \llbracket \overline{E}, E' \rrbracket$$

Figure 1.4: Translation into Annotations 1

$$\llbracket x(\overline{T_1}) :: T_2 \rrbracket = \llbracket \overline{T_1} \rrbracket \rightarrow \llbracket T_2 \rrbracket$$

$$\llbracket x(\overline{T_1}) :: T_2 \text{ when } \overline{\alpha : T_3} \rrbracket = \llbracket \overline{T_1} \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}$$

Figure 1.5: Translation into Annotations 2

$$\llbracket \overline{E}, x(\overline{T'_1}) :: T'_2 \rrbracket = \begin{cases} S \text{ and } \llbracket \overline{T'_1} \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} & \text{if } \llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ \llbracket \overline{E} \rrbracket \text{ and } \llbracket \overline{T'_1} \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases}$$

Figure 1.6: Translation into Annotations 3

$$\llbracket \overline{E}, x(\overline{T'_1}) :: T'_2 \text{ when } \overline{\alpha' : T'_3} \rrbracket = \begin{cases} S \text{ and } \llbracket \overline{T'_1} \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{if } \llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ \text{when } \overline{\alpha : \llbracket T_3 \rrbracket}, \overline{\alpha' : \llbracket T'_3 \rrbracket} & \text{and } \overline{\alpha} \cap \overline{\alpha'} = \emptyset \\ S \text{ and } \llbracket \overline{T'_1} \rrbracket \rightarrow \llbracket T'_2 \rrbracket [\alpha' := \beta'] & \text{if } \llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket} \\ \text{when } \overline{\alpha : \llbracket T_3 \rrbracket}, \overline{\beta' : \llbracket T'_3 \rrbracket} & \text{and } \overline{\alpha} \cap \overline{\alpha'} \neq \emptyset \\ \llbracket \overline{E} \rrbracket \text{ and } \llbracket \overline{T'_1} \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \overline{\alpha' : \llbracket T'_3 \rrbracket} & \text{otherwise} \end{cases}$$

Figure 1.7: Translation into Annotations 4

Figure 1.5 handles a single clause, $\$ \llbracket E \rrbracket$, in isolation, which provides the base case for the expression-level translation. An unguarded clause $x(\overline{T_1}) :: T_2$ is mapped to the function type $\llbracket \overline{T_1} \rrbracket \rightarrow \llbracket T_2 \rrbracket$: argument types and the return type are each translated pointwise, and the divider in between them, $::$, becomes an arrow, $\rightarrow$ which is used for any other function types in Elixir Type notation. A guarded clause $x(\overline{T_1}) :: T_2 \text{ when } \overline{\alpha : T_3}$ is handled analogously, with the `when` clause preserved verbatim after translating each bound type T_3 . The type variable names α are carried over unchanged; they now range over Elixir Type values but retain the same identifiers as the source.

Figure 1.6 extends the translation to a sequence whose new clause carries no `when` guard. The input is a pair $(\overline{E}, x(\overline{T'_1}) :: T'_2)$ where $\overline{E}$ is a sequence of preceding TypeSpec expression clauses. Two cases arise depending on whether the translated sequence of preceding expressions carries a `when` clause. If $\llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}$, the new clause is appended as an intersection with `and`, and the existing `when` guard is attached to it at the tail of it, so that the type variables scope over the earlier clauses as well as the new one. If the prefix carries no `when` clause, the new clause is simply conjoined: $\llbracket \overline{E} \rrbracket \text{ and } \llbracket \overline{T'_1} \rrbracket \rightarrow \llbracket T'_2 \rrbracket$. This case reflects the fact that an unguarded new clause introduces no new type variables of its own, so no scoping adjustment is needed.

Figure 1.7 handles appending a new `when` clause when a new expression carries a `when` guard, $\overline{\alpha' : T'_3}$. Three sub-cases arise based on whether the existing translated expression carries a `when` guard and, if so, whether the variable names overlap with the incoming clause. First, if $\llbracket \overline{E} \rrbracket \approx S \text{ when } \overline{\alpha : \llbracket T_3 \rrbracket}$ and the two sets of type

variables are disjoint ($\overline{\alpha} \cap \overline{\alpha'} = \emptyset$), the guards can be merged directly. In other words, the names of type variables from the new `when` clause are all distinct from those in the existing clause. Hence, the new clause is appended with `and` and all type variable bindings from both guards are collected into the merged `when` clause. Second, if the two sets of type variables overlap ($\overline{\alpha} \cap \overline{\alpha'} \neq \emptyset$), the new clause's type variables are renamed to fresh names $\overline{\beta'}$ before merging, so that no variable from the existing scope is accidentally captured; the renamed clause is then appended under the combined `when` clause. Third, if the existing expressions carry no `when` clause at all, the new clause is simply appended together with its own guard: $\llbracket \overline{E} \rrbracket$ and $\llbracket \overline{T_1'} \rrbracket \rightarrow \llbracket \overline{T_2'} \rrbracket$ when $\overline{\alpha'} : \llbracket \overline{T_3'} \rrbracket$. These three cases together ensure that multi-clause annotations with heterogeneous guard structures are always translated to a single, well-scoped Elixir Type signature.

1.3.2 Translation of Basic Types

$\llbracket \alpha \rrbracket = \alpha$
 $\llbracket T_1 \mid T_2 \rrbracket = \llbracket T_1 \rrbracket \text{ or } \llbracket T_2 \rrbracket$
 $\llbracket \text{any}() \rrbracket = \text{term}()$
 $\llbracket \text{none}() \rrbracket = \text{none}()$
 $\llbracket \text{atom}() \rrbracket = \text{atom}()$
 $\llbracket :k \rrbracket = :k$
 $\llbracket \text{pid}() \rrbracket = \text{pid}()$
 $\llbracket \text{port}() \rrbracket = \text{port}()$
 $\llbracket \text{reference}() \rrbracket = \text{reference}()$
 $\llbracket \text{float}() \rrbracket = \text{float}()$
 $\llbracket \text{integer}() \rrbracket = \text{integer}()$
 $\llbracket \text{neg_integer} \rrbracket = \text{infty}..-1$
 $\llbracket \text{non_neg_integer} \rrbracket = 0..\text{infty}$
 $\llbracket \text{pos_integer} \rrbracket = 1..\text{infty}$
 $\llbracket \text{tuple}() \rrbracket = \text{tuple}()$
 $\llbracket \{\overline{T}\} \rrbracket = \{\llbracket \overline{T} \rrbracket\}$
 $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$
 $\llbracket [] \rrbracket = \text{empty_list}()$
 $\llbracket \text{nonempty_maybe_improper_list}(T_1, T_2) \rrbracket = \text{non_empty_list}(\llbracket T_1 \rrbracket, \llbracket T_2 \rrbracket)$

Figure 1.8: Translation in Types

$$\begin{aligned}
\llbracket \langle \rangle \rrbracket &= \text{bitstring}() \\
\llbracket \langle _ :: n \rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket \langle _ :: _ * n \rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket \langle _ :: n, _ :: _ * n \rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket (\bar{T} \rightarrow T) \rrbracket &= (\llbracket \bar{T} \rrbracket \rightarrow \llbracket T \rrbracket) \\
\llbracket (\dots \rightarrow T) \rrbracket &= \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ (\rightarrow \llbracket T \rrbracket) \text{ or } \dots \text{ or } \underbrace{(\text{none}(), \dots, \text{none}()) \rightarrow \llbracket T \rrbracket}_{255 \text{ times}} & \text{otherwise} \end{cases} \\
\llbracket n \rrbracket &= n..n \\
\llbracket n..n' \rrbracket &= n..n'
\end{aligned}$$

Figure 1.8: Translation in Types (cont.)

Most entries in Figure 1.8 are structural or notational translations. Type variables, union, `none()`, the atom types `atom()` and `:k`, the process handle types `pid()`, `port()`, and `reference()`, the numeric base types `float()` and `integer()`, the untyped and component-wise tuple forms, the empty-list constructor `[]`, the non-empty improper list `nonempty_maybe_improper_list(T1, T2)`, fixed-arity function types, and integer ranges `n..n'` all translate component-wise, with at most a renaming of the constructor. The following cases deviate from a purely direct mapping and warrant comment.

Top type. TypeSpecs names the top type `any()`, while Elixir Types names it `term()`. The rule $\llbracket \text{any}() \rrbracket = \text{term}()$ is therefore a pure renaming with identical semantics. Note that TypeSpecs already defines `term()` as an alias for `any()` in its built-in type table, so the two names are interchangeable at the source level; the translation simply adopts the Elixir Type choice of name.

Ranged integer subtypes. TypeSpecs provides three ranged shorthands for common half-bounded integer subsets: `neg_integer()`, `non_neg_integer()`, and `pos_integer()`. Elixir Types has no exact counterparts; it represents every integer subset uniformly as a range. The translation maps these to `infty..-1`, `0..infty`, and `1..infty` respectively, where `infty` is the special sentinel for the unbounded endpoint of an integer range. However, the interval types are only for internal developer level use; the users will not see these types but all as `integer()`. Valim addressed this as trade-off between precision and simplicity, and this is a design choice for simplicity in practice Valim (2026)

Integer singletons. An integer literal `n` in TypeSpecs denotes a singleton type containing that value alone. Elixir Types has no standalone singleton notation: every integer type is a range. The rule $\llbracket n \rrbracket = n..n$ represents the singleton as the degenerate range whose two endpoints coincide. Nevertheless, same as the ranged integer subtypes, this is also not available for users and we later translate them all to `integer()`.

Bitstring types. TypeSpecs expresses precise bit-length constraints through the forms `⟨_ :: n⟩`, `⟨_ :: _ * n⟩`, and `⟨_ :: n, _ :: _ * n⟩`. However, this precision is rarely needed in practice; the bit-length forms are hardly used. Hence, Elixir Types exposes only two bitstring constructors: `binary()` for byte-aligned bitstrings (bit-lengths that are multiples of 8) and `bitstring()` for arbitrary bitstrings. The translation collapses each parametric form by testing divisibility by 8: when $n \bmod 8 = 0$ the bitstring is byte-aligned and the result is `binary()`; otherwise the result is `bitstring()`. In both cases the exact bit-length information is discarded, so the translated type is a conservative over-approximation of the source type. The unconstrained form `⟨⟩` translates to `bitstring()` as a direct renaming, since it denotes any bitstring with no size restriction.

Variable-arity function types. The TypeSpec form `(... -> T)` denotes a function of unspecified arity returning `T`. Set-theoretically, that becomes a function type that contains all the possible function types. Elixir Types

requires each function type to have a fixed arity, so there is no direct counterpart. Two sub-cases arise depending on the return type. When $\llbracket T \rrbracket \approx \text{term}()$, any function of any arity satisfies the specification and the result is $\text{fun}()$, the top function type in Elixir Types. When the return type is more specific, the translation expands to a union over all fixed arities from 0 to 255 — the maximum arity permitted by the BEAM — using $\text{none}()$ as the placeholder argument type for each parameter position. This expansion is the only place in the basic type translation where the size of the output grows proportionally to the maximum arity. In practice, $(\dots \rightarrow T)$ arises most often through the $\text{fun}()$ built-in, which TypeSpecs desugars to $(\dots \rightarrow \text{any}())$; that instance falls into the first sub-case and produces simply $\text{fun}()$.

1.3.3 Translation of Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or $\text{if_set}(T)$. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by Empty that maps all the non-singleton key types to $\text{not_set}()$.

First we define the translation of single field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $[\bigvee_{i=1}^n K_i \Rightarrow \tau]$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 1.9:

$$\begin{aligned} \llbracket \text{required}(T) \Rightarrow T' \rrbracket &= \begin{cases} [:k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx :k \\ [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \text{ and } \llbracket T \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases} \\ \llbracket \text{optional}(T) \Rightarrow T' \rrbracket &= \begin{cases} [\bigvee_{i \in I} K_i \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \approx \bigvee_{i \in I} K_i \\ \text{undefined} & \text{otherwise} \end{cases} \end{aligned}$$

where all K_i 's in I are pairwise distinct.

Figure 1.9: Translation in Field Types

After obtaining types in Elixir Types we can define merge function. Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $K \Rightarrow \tau$ of L_2 . Formally:

$$\begin{aligned} L_1 \oplus [] &= L_1 \\ L_1 \oplus [K \Rightarrow \tau, L_2] &= [K \Rightarrow \tau, L_1] \oplus L_2 && \text{if } K \notin L_1 \\ L_1 \oplus [K \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } K \in L_1 \\ L_1 \oplus [:k \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } \text{atom}() \in L_1 \end{aligned}$$

Figure 1.10: Merge in Field Types

We then finally define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{ F_1, \dots, F_n \} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible key types), then we have $\llbracket \% \{ F_1, \dots, F_n \} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty})$.

1.4 Total Approximating Translation of TypeSpecs into Elixir Types

The translation defined so far is partial since some TypeSpec types do not have their exact corresponding types in Elixir Types. This is not a weakness of the Elixir Type system, but rather a consequence of the design choices made for Elixir Types, to keep it as simple as possible while still being useful. However, it is possible to define a total translation from types in TypeSpecs to ones in Elixir Types by approximating the original types in TypeSpecs. Total translation is useful in the context of migration, when we need to ensure that every type in TypeSpecs is represented in Elixir Types even approximately.

More precisely, the previous translations are *undefined* in the cases of translating field types; when $\llbracket T \rrbracket$ in $\llbracket \text{required}(T) \Rightarrow T' \rrbracket$ or $\llbracket \text{optional}(T) \Rightarrow T' \rrbracket$ is not equivalent to a disjunction of pairwise distinct key types.

In the rest of this section we give a conservative extension of the translation defined so far that is total, i.e., it is defined for every type in TypeSpecs. The extension consists in defining approximations for the cases where the previous translation is undefined.

1.4.1 Approximation of Field Types

The merging case with conjunctive left-hand sides does not only perform merge function but also conduct approximation of types in TypeSpecs that are smaller than key types in Elixir Types. That is because while merge must take original intention of types in TypeSpecs into account, if we compare left-hand side types after blunt individual type approximation, in particular over-approximation, the transition could omit subtle type information.

Keeping that in mind, we can safely write the promotion rule with approximation from TypeSpec types that are *undefined* to key types in Elixir Types. Which rule essentially lets an approximated type carry its original type so that the merge function can use the extra information to minimise the loss of the original semantics in translation.

To begin with, we approximate the left-hand side of a field type $\llbracket T \rrbracket$ to K , i.e., to its smallest supertype in Elixir key types. Then superscript the original type information to the approximated type, $K^{\llbracket T \rrbracket}$. In other words, $K^{\llbracket T \rrbracket}$ is the key type promoted from $\llbracket T \rrbracket$. This is important for the merge function when applied to approximated field types in terms of comparison. Consider merging $1..2 \Rightarrow \text{integer}()$ and $3..4 \Rightarrow \text{float}()$. Although the key types are disjunctive, if we approximate $1..2$ and $3..4$ without the original information, the key type comparison merely becomes of $\text{integer}()$ with $\text{integer}()$, resulting in completely overwriting one for another. Whereas, with our promotion rule, the merge function can use the preserved information for the comparison keeping both right-hand side types in the given example. Hence, to rewrite the translation of field types after applying the promotion rule:

$$\llbracket \text{required}(T) \Rightarrow T' \rrbracket = \begin{cases} [:k^k \Rightarrow \llbracket T' \rrbracket] & \text{if } \llbracket T \rrbracket \approx :k \\ [\bigvee_{i=1}^n :k_i^{k_i} \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{if } \llbracket T \rrbracket \not\approx :k \text{ and } \llbracket T \rrbracket = \bigvee_{i=1}^n :k \vee T_0 \\ [\bigvee_{\{K \in \mathbb{K}^+ \mid \llbracket T_0 \rrbracket \cap K \neq \emptyset\}} K^{\llbracket T_0 \rrbracket} \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{and } T_0 \cap \text{atom}() = (\emptyset \text{ or } \text{atom}()) \end{cases}$$

$$\llbracket \text{optional}(T) \Rightarrow T' \rrbracket = \begin{cases} [\bigvee_{i=1}^n :k_i^{k_i} \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{where } \llbracket T \rrbracket = \bigvee_{i=1}^n :k \vee T_0 \\ [\bigvee_{\{K \in \mathbb{K}^+ \mid \llbracket T_0 \rrbracket \cap K \neq \emptyset\}} K^{\llbracket T_0 \rrbracket} \Rightarrow \text{if_set}(\llbracket T' \rrbracket)] & \text{with } T_0 \cap \text{atom}() = (\emptyset \text{ or } \text{atom}()) \end{cases}$$

where $\mathbb{K}^+$ is a set of all the key types except singleton atom types, i.e., $\mathbb{K}^+$ is the set $\{\text{atom}(), \text{pid}(), \text{port}(), \text{reference}(), \text{float}(), \text{integer}(), \text{bitstring}(), \text{binary}(), \text{tuple}(), \text{open_map}(), \text{fun}(), \text{list}()\}$

Figure 1.11: Approximated Translation in Field Types

From this approximated translation, the merge function extends its definition further with the extra information. In the following definition, we substitute superscripted $\llbracket T \rrbracket$ with s for convenience.

$$\begin{aligned}
[L_1] \oplus [] &= [L_1] & (M1) \\
L_1 \oplus [K^s \Rightarrow \tau, L_2] &= [K^s \Rightarrow \tau, L_1] \oplus L_2 & \text{if } K \notin L_1 & (M2) \\
L_1 \oplus [K^s \Rightarrow \tau, L_2] &= L_1 \oplus L_2 & \text{if } K^{s'} \in L_1 \text{ and } K \cap s \leq K \cap s' & (M3) \\
L_1 \oplus [K^s \Rightarrow \tau, L_2] &= [K^{s \vee s'} \Rightarrow \tau \text{ or } \tau', L_1 \setminus K^{s'}] \oplus L_2 & \text{if } K^{s'} \Rightarrow \tau' \in L_1 \text{ and } K \cap s \not\leq K \cap s' & (M4) \\
L_1 \oplus [:k^s \Rightarrow \tau, L_2] &= L_1 \oplus L_2 & \text{if } \text{atom}()^{s'} \in L_1 \text{ and } :k \leq s' & (M5) \\
L_1 \oplus [:k^s \Rightarrow T, L_2] &= [:k^{s'} \Rightarrow T', L_1 \setminus :k^{s'}] \oplus L_2 & \text{if } :k^{s'} \Rightarrow \text{if_set}(T') \in L_1 & (M6)
\end{aligned}$$

we use the following notation: (1) $K^s \in L$ if $\exists \tau$ such that $K^s \Rightarrow \tau \in L$; (2) $K \in L$ if $\exists s, \tau$ such that $K^s \Rightarrow \tau \in L$; and (3) $L \setminus K$ for $L \setminus (K^s \Rightarrow \tau)$ if $\exists s, \tau$ such that $K^s \Rightarrow \tau \in L$.

Figure 1.12: Merge in Approximated Field Types

While the rest of rules are modified to operate disjoint merge cases, both Rule (M1) and (M2), two cases describing the complete disjoint, trivially behave the same as merging field types without the promotion rule.

The disjoint merge cases are divided into two groups: Rule (M3) and (M4), and Rule (M5) and (M6). The former group defines the cases when the approximated key type of a merging field type is not a singleton atom key type. Conversely, the latter defines others when the key type is a singleton atom type.

In Rule (M3) and (M4), if a key type match is found in the merged list, we intersect its key type with its original type, and intersect the key type with another from the merging field type, in order to compare the result types and find their subtype relationship. When the subtype relationship is met, the merging field type is voided, which is the definition of Rule (M3). Otherwise, we append the merging field type to the list with two union operations: a unification of superscripted original left-hand types from both fields; and that of right-hand types. Since Rule (M4) append a newly merged field to the list, the field type found in the list is eradicated in order not to carry duplicated fields.

The bottom line of Rule (M5) is as same as Rule (M3). It describes that a merging field type with any singleton atom key type is overwritten by a field type with $\text{atom}()$ as its key type. This rule applies because it is always true that $s = :k$ in $:k^s$, and $s' = \text{atom}()$ in $\text{atom}()^{s'}$.

The approach of Rule (M6) is different from the previous rules. This rule describes when attempted merging a required field type. If the merged list has an optional field type with the same singleton atom type as its key type, this field turns into a required field type and the merging field is removed.

We revise the field type translation to $\llbracket \% \{ F_1, \dots, F_n \} \rrbracket = \text{erase}(\text{map}(\oplus_{i=1}^n \llbracket F_i \rrbracket))$, where *erase* eradicates the superscripted original types and only leaves the allowed Elixir key types. Thus, by merging Empty to that end of merges, we obtain a saturated map, $\llbracket \% \{ F_1, \dots, F_n \} \rrbracket = \text{erase}(\text{map}(\oplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty}))$.

In addition, although we take every map type as closed, $\text{map}()$ in TypeSpecs is in fact an open map, i.e., $\% \{ \text{optional}(\text{any}()) \Rightarrow \text{any}() \}$. Which is expanded to a list of every key type mapping to $\text{term}()$ when translated, hence effectively overwriting all the following field types. This is all solid but listing every key type to the top type is definitely burdensome when displaying it. Then we denote this with triple dots in a map type: $\% \{ \dots \}$; and also with preceding required field type: $\% \{ \dots, :k \Rightarrow T \}$.

Chapter 2

Implementation and Analysis

This chapter demonstrates the translation we have formalised in the previous chapter.

We first build a prototype that translates TypeSpecs based on Elixir’s abstract syntax tree (AST) and produces Elixir Types in which form customised to be consumed by the new compiler (version 1.21.0-dev). This prototype also involves collecting and translating user-defined types or remote types that are defined by directives `@type`, `@opaque`, and `@typep` in order to expand them in TypeSpecs. The following section runs Dialyzer and the new compiler’s typechecker on various codebases that are used in the real world. Which process produces the raw dataset. Then, we analyse the raw dataset, examining the warning messages and success rate with quality. The final analysis eventually helps building a feasible dataset that we can use for applying to large language models in the next part.

2.1 Implementation of theory

The implementation is organised into three layered components. The first layer is the translator, which realises the formal semantics of the type translation process. The second layer is the migrator, which uses the translator to resolve user-defined types and generate concrete outputs. The third layer is the dataset producer, which automates translation across projects and records typecheck outcomes from Dialyzer and the new typechecker.

Environment and reproducibility. The entire pipeline was run locally on a single machine—an Apple M1 (macOS) with 8 GB of RAM. Because the two analyses require incompatible preparation of the same source, the corpus is materialised as two independent copies of every project: a *translation–typecheck* copy, whose source the migrator (run under Elixir 1.19.5 on Erlang/OTP 28.1) annotates with the injected Elixir Types and `@assert_type` form directives and which the typechecker then compiles with the customised 1.21.0-dev compiler; and a *Dialyzer* copy, left otherwise source-unmodified and analysed under the same Elixir 1.19.5. Each copy fetches its own dependencies with `mix deps.get`, so both type resolution and Dialyzer see real dependency sources, and a project whose dependencies fail to fetch is skipped from that copy. Recording this build closure matters for reproducibility: which remote types and cross-module callees resolve depends on what each copy fetched, hence, a fraction of the tool disagreements analysed later are properties of the environment rather than of the code.

2.1.1 Translator

The translator is the core component that converts extracted AST of TypeSpecs into the internal type representation used by our prototype. It is implemented primarily in `Migrator.Translator.Utils` with support from `Migrator.Translator.Approximator`. The translation pipeline is divided into three phases:

1. *AST acquisition*
2. *Type normalisation*
3. *Internal translation and approximation*

AST Acquisition

In the first phase, source files are parsed into quoted Elixir terms using `Code.string_to_quoted!/1`, wrapped by a function `safe_string_to_quoted/1` for exceptions. This wrapper catches `SyntaxError`, `TokenMissingError`, and `MismatchedDelimiterError` exceptions and logs them as warnings, ensuring that only well-formed ASTs proceed to the following phases and that malformed files are gracefully skipped without aborting the overall translation run.

Type Normalisation

In the second phase, the translator normalises the `TypeSpec` syntax into an internal form; in principle, this is to remove unnecessary information, such as meta data, when it comes to translation. The `Utils.parse/2` function rewrites high-level constructs into uniform AST patterns expressed only with the primitive forms of the `TypeSpec` syntax specification. Elixir AST has a form of a triple-element tuple: {an atom or another tuple, a list of double-element tuples with metadata (e.g., line numbers), a list of arguments}. We keep this syntax at this phase while emptying the second element whenever possible and converting the built-in types and syntactic sugars into the primitive (basic) forms by recursively exploring argument types.

Several expansion rules are worth noting. Numeric subtypes are reduced to range notation: `neg_integer()` becomes `:infty..-1`, `non_neg_integer()` becomes `0..:infty`, and `pos_integer()` becomes `1..:infty`, where `:infty` is a sentinel for an unbounded endpoint. Built-in collection shorthands are similarly expanded: `binary()` is rewritten into a bitstring pattern `«_:_*8»`, `charlist()` becomes `list(char())`, and `keyword(T)` unfolds into an explicit list-of-pairs form `[] | nonempty_maybe_improper_list({atom(), T}, [])`. The `struct()` type is expanded into a generic struct pattern carrying a sentinel key `__struct_top__` alongside an open map of atom-keyed fields, allowing later code to distinguish typed structs from the unqualified `struct()` type, which the sentinel marks as the top of the struct hierarchy.

Notice that we intentionally parses `binary()` to `«_:_*8»` even though later it is translated back to `binary()` again. That is because the purpose of the `Utils.parse/2` function is to deliver type syntax that only consists of primitive `TypeSpec` forms to `Utils.translate/2` and to rigorously show the process of translation. Therefore, we take every step explicitly putting aside performance details.

Module references are also resolved during this phase. Any occurrence of `__MODULE__` inside a qualified type name is replaced with the string name of the enclosing module so that all remote type references carry a fully lengthened, context-independent identifier. Guard type variables are tagged with a special context marker `:__guard_type_variable__`, and user-defined type variables appearing in parametric type definitions are tagged with `:__user_type_variable__`, keeping them syntactically distinct even though they carry the same name as basic or built-in types.

Internal Type Representation

In the third phase, normalised type trees are converted into our internal type representation by the `translate/2` function. The representation is a tree of Elixir terms, i.e., atoms for ground types and tagged tuples for composite types, chosen so that types can be compared structurally with only essential information, accumulated in lists, and pattern-matched efficiently. We let $\mathbb{T}$ to range over the internal representation of Elixir terms in this section.

Ground types are represented as plain atoms:

```
:term, :none, :atom, :integer, :float, :pid, :port,  
:reference, :binary, :bitstring, :tuple, :open_map, :fun, :empty_list
```

Composite types are tagged tuples whose first element identifies the constructor:

Union types are stored as a right-leaning binary tree:

$$T_1 \mid T_2 \mapsto \{:\text{union}, \{T_1, T_2\}\}$$

so that a union of n alternatives becomes a spine of nested `:union` nodes. Two helpers, `flatten_from_union_to_list` and `unflatten_from_list_to_union`, convert between the tree form and a flat list, which is the form used internally by the approximation algorithms.

Integer ranges carry their endpoints directly:

$$n_1..n_2 \mapsto \{:\text{interval}, \{n_1, n_2\}\}$$

where either endpoint may be `:infy` to represent an unbounded half-line. Integer singleton types (e.g., the literal 97) are encoded as unit intervals `\{:\text{interval}, \{97, 97\}\}`.

Atom singletons are stored as `\{:\text{atom}, :k\}` for a literal atom `:k`. When an atom appears as a map key, its requiredness is tracked via the wrappers `\{:\text{atom\_req}, :k\}` for required keys and `\{:\text{atom\_opt}, :k\}` for optional keys.

Maps have two variants. An open map (written `\%{\text{optional}(\text{any}()) \Rightarrow \text{any}()}` in `TypeSpecs`) is the ground type `:open_map` and is the only open map type from `TypeSpec`. A closed map at first carries its field list:

$$\{:\text{closed_map}, [\{\mathbb{T}_{k_1}, \mathbb{T}_{v_1}\}, \dots, \{\mathbb{T}_{k_1}, \mathbb{T}_{v_n}\}]\}$$

where each key $\mathbb{T}_{k_i}$ is itself a type descriptor showing whether or not a field is mandatory (e.g., `\{:\text{atom\_req}, :name\}` or `:integer`) and each value $\mathbb{T}_{v_i}$ is a translated type. Then these translated types are approximated by `Approximation.promote()` and `Approximation.map()` functions to eventually evaluate the left side types to be in the range of Elixir Types' key types K . Which process involves wrapping optional fields whose value type is determined by merging with `\{:\text{if\_set}, \mathbb{T}\}`, distinguishing them from mandatory fields. We further explain approximation implementation in the next subsection.

Structs share the field-list structure of closed maps but additionally record the module name:

$$\{:\text{struct}, \{\text{ModuleName}, [\{:\text{atom_req}, \mathbb{T}_{v_1}\}, \dots]\}\}$$

A special sentinel label `:__struct\_top__` is used to represent the bare `struct()` type, which matches any struct without naming a specific module.

Lists have three forms: `:empty_list` for `[]`, and

$$\{:\text{non_empty_list}, \{\mathbb{T}_{\text{content}}, \mathbb{T}_{\text{termination}}\}\}$$

for a non-empty (possibly improper) list. The termination type is always `:empty_list` for proper lists, but improper lists carry a termination type that is none other than `:empty_list`.

Functions distinguish fixed-arity from variable-arity forms:

$$\begin{aligned} (\mathbb{T}_{in_1}, \dots, \mathbb{T}_{in_n} \rightarrow \mathbb{T}_{out}) &\mapsto \{:\text{fun}, [\{\mathbb{T}_{in_1}, \dots, \mathbb{T}_{in_n}\}, \mathbb{T}_{out}]\} \\ (\dots \rightarrow \mathbb{T}_{out}) &\mapsto \{:\text{fun}, \{:\text{all_arity}, \mathbb{T}_{out}\}\} \end{aligned}$$

A bare `fun()` (any function, any output) translates to the ground type `:fun`.

User-defined and remote types carry references that are resolved in later phases:

$$\begin{aligned} \text{my_type}() &\mapsto \{:\text{user_type}, :\text{my_type}\} \\ \text{my_type}(t) &\mapsto \{:\text{user_type}, \{:\text{my_type}, [t]\}\} \\ \text{Mod.t}() &\mapsto \{:\text{remote_type}, \{[\text{Mod}], :t\}\} \end{aligned}$$

Type variables, t , introduced by parametric type definitions are represented as `\{:\text{user\_type\_var}, name\}`. This is distinguished from guard-constrained variables represented as `\{:\text{guard\_type\_var}, name\}` which is used in `when` clause.

Approximation

Because the new type system imposes constraints that `TypeSpecs` does not, some type expressions cannot be translated exactly. The `Migrator.Translator.Approximator` module implements a two-step approximation procedure that first promotes individual types into category groups that are key types of Elixir Types and then merges overlapping fields judging by their key types' subtype relationship.

Union key splitting. Before promotion or merging takes place, the `field_translator` closure (anonymous function) in `translate/2` preprocesses each field of a map literal. If a field's key type is a union (for example, `optional( $\mathbb{T}_{K_1} \mid \mathbb{T}_{K_2}$ ) =>  $\mathbb{T}_V$ )`, the closure flattens the union and emits one field per branch: `{ $\mathbb{T}_{K_1}$ , [ $\mathbb{T}_V$ ]}` and `{ $\mathbb{T}_{K_2}$ , [ $\mathbb{T}_V$ ]}`. Same as the non-literal types all the field types whose key types are split from a union type are treated as optional including atom literal types which hence tagged as `{:atom_opt, :k}`.

Promotion. The `Approximator.promote/1` function then wraps each field's key into a pair `{category, [original_key]}`, where `category` is the broad Elixir type the key belongs to, and `[original_key]` carries the precise key type for later subtype checking. The mapping is:

```

{:closed_map, _}  => {:open_map, [ $\mathbb{T}_{original\_key}$ ]}
{:struct, _}     => {:open_map, [ $\mathbb{T}_{original\_key}$ ]}
{:tuple, _}      => {:tuple, [ $\mathbb{T}_{original\_key}$ ]}
{:empty_list, _} => {:list, [ $\mathbb{T}_{original\_key}$ ]}
{:non_empty_list, _} => {:list, [ $\mathbb{T}_{original\_key}$ ]}
{:fun, _}        => {:fun, [ $\mathbb{T}_{original\_key}$ ]}
{:interval, _}   => {:integer, [ $\mathbb{T}_{original\_key}$ ]}
{:atom_req, :k}  => {:atom, [{:atom_req, :k}]}
{:atom_opt, :k}  => {:atom, [{:atom_opt, :k}]}

```

Ground types that have no finer form (e.g. `:atom`, `:integer`) fall through and are wrapped as `{ $\mathbb{T}_{ground}$ , [ $\mathbb{T}_{ground}$ ]}`. The purpose of this two-level structure is to allow the merge step to first perform a cheap category equality check before invoking the more expensive `unify_types` operation.

Map field merging. As we have shown in the field type approximation specification, it is possible for two fields to have overlapping key types within a single map type. For example, `%{optional(atom()) => string(), required(:foo) => integer() }` yields two fields whose keys, `:atom` and `{:atom_req, :foo}`, both promote to the category `:atom`. The `Approximator.map/1` function folds the promoted field list into an accumulator L , and for each incoming field $F_{new} = \{C_{new}, O_{new}, V_{new}\}$ it proceeds as follows with cascading strategy:

1. **Category lookup.** The algorithm searches L for a field $F_L = \{C_L, O_L, V_L\}$ whose category C_L equals C_{new} (or is `:term`) in order to judge if checking a subtype relationship is necessary.
2. **Original-key subtype check.** If a matching category is found, `unify_types/1` is called on the pair of original keys $\{O_{new}, O_L\}$ to determine whether the incoming original keys are subtypes of the existing ones.
3. **Merging decision.** Four cases arise from the three Boolean flags (category is matched, category is generic `atom()`, and original keys are a subtype):
 - Category matched, not the generic atom category, and $O_{new} \leq O_L$: the incoming field is entirely subsumed; discard F_{new} and leave L unchanged.
 - Category matched, not the generic atom category, and $O_{new} \not\leq O_L$: the keys are disjoint or partially overlap but neither subsumes the other; merge the value lists by concatenation and update the original key list.
 - Category matched, is the generic atom category, and $O_{new} \leq O_L$: if no residual original keys remain after unification, absorb F_{new} into the existing `:atom` field; otherwise retain L unchanged.
 - Category not matched: F_{new} introduces a complete disjoint key shape and is appended to L as a new entry.
4. **Right-hand value merging.** Once all fields are processed, each accumulated value list is deduplicated, subsumed entries are removed via `unify_types/1`, and the survivors are joined into a union by applying `unflatten_from_list_to_union/1`.
5. **Optionality wrapping.** Any value type of a field whose key type is not a required atom literal is wrapped in `{:if_set, ...}`, marking it as a field that may be absent in a concrete closed-map value.

Type unification and subtype checking. The core predicate underlying both promotion and map merging is `unify_types/1`, which takes a pair of original types, merges any elements that stand in a subtype relationship to a element from the accumulating list, and returns the updated list together with a Boolean flag indicating whether the merging type was entirely subsumed by the second. Its case analysis covers every pairing of the internal type constructors. Several cases are particularly noteworthy.

Integer intervals. Given two intervals $[n_1, n_2]$ and $[m_1, m_2]$ (with `:infty` representing an unbounded endpoint), the helper `merge_types` covers ten cases in total: one disjoint check followed by the nine non-disjoint cases that arise from the 3×3 comparison of the left endpoints (n_1 vs. m_1) and right endpoints (n_2 vs. m_2). The disjoint check is performed first via `Range.disjoint?/2`; the remaining nine cases group into three outcomes:

- **Disjoint:** retain both intervals; not a subtype.
- $[n_1, n_2] \subseteq [m_1, m_2]$ (4 cases: $n_1 \geq m_1$ and $n_2 \leq m_2$, including the identical case): discard $[n_1, n_2]$; is a subtype.
- $[m_1, m_2] \subsetneq [n_1, n_2]$ (3 cases: $n_1 \leq m_1$ and $n_2 \geq m_2$, excluding identical): retain $[n_1, n_2]$ and discard $[m_1, m_2]$; not a subtype.
- **Partial overlap** (2 cases: neither interval contains the other): approximate by the smallest enclosing interval $[\min(n_1, m_1), \max(n_2, m_2)]$; not a subtype.

The subtype flag is `true` only when the *first* operand is contained in the second; when the second is the smaller interval, the flag is `false` even though one interval subsumes the other. Unbounded endpoints are handled by substituting a finite sentinel value (one below the global minimum or one above the global maximum of the four bounds) for the purpose of the disjoint and containment tests, then restoring `:infty` in the result.

Atom keys. The required/optional distinction interacts with the generic `:atom` type through asymmetric rules: $\{:\text{atom_req}, :k\}$ and $\{:\text{atom_opt}, :k\}$ is a subtype of `:atom` (a specific required key is a special case of any atom), but the converse does not hold. Two distinct required singletons $\{:\text{atom_req}, :k_1\}$ and $\{:\text{atom_req}, :k_2\}$ are never in a subtype relation with each other; instead they are accumulated as separate candidates. A required singleton $\{:\text{atom_req}, :k\}$ is a subtype of $\{:\text{atom_opt}, :k\}$ (with the same key) because a field that is always present trivially satisfies the weaker optionality constraint.

Non-empty lists. Two non-empty list types $\{:\text{non_empty_list}, \{T_1, T_1\}\}$ and $\{:\text{non_empty_list}, \{T_1, T_2\}\}$ are unified element-wise: the unification of content types and that of termination types are independent. If both pairwise results are subtypes, the first list type is subsumed; otherwise a single merged non-empty list is produced by rebuilding the content and termination types from their respective unification results.

Functions. Function subtyping first checks if the arities differ; in which case the two function types are kept separate. If the arities match then it follows the contravariant and covariant rules for a list of inputs and an output, respectively. For two fixed-arity functions $f_1 : (\bar{T}_1 \rightarrow T'_1)$ and $f_2 : (\bar{T}_2 \rightarrow T'_2)$ with the same arity, f_1 is a subtype of f_2 if and only if $\bar{T}_2 \leq \bar{T}_1$ (inputs reversed) and $T'_1 \leq T'_2$. A variable-arity function $\{:\text{fun}, \{:\text{all_arity}, T'\}\}$ is treated as a supertype of any same output fixed arity function: if the output types unify, the fixed arity variant is absorbed.

Structs. Two struct types are unified field-by-field: for each field of the first struct, the algorithm checks whether a matching field (by key and value subtyping) exists in the second. The special sentinel struct $\{:\text{struct}, \{:\text{__struct_top_}, _ \}\}$ representing the unqualified `struct()` type is always a supertype of any named struct, so any specific struct is subsumed by it.

2.1.2 Migrator

The migrator builds on the translator by resolving user-defined types and generating output artefacts. Its pipeline consists of three stages:

1. *User-defined type translation*
2. *Spec translation*
3. *Output construction*

The first stage is implemented by the `Migrator.TypeTranslator` module. It scans input files for `@type`, `@opaque`, and `@typep` declarations and extracts each definition into a module-local table. Type variables are marked explicitly, then definitions are parsed and recursively expanded so that user-defined types may later be substituted during the Elixir Types construction stage. Recursive type definitions are detected by maintaining a stack of type names currently being expanded; if a name reappears on the stack, the definition is replaced by `:dynamic` to avoid non-termination. Expansion is iterated to a fixed point so that transitive references through chains of user-defined types are fully resolved.

The second stage is implemented by the `Migrator.SpecTranslator` module, which we have described in Subsection 2.1.1.

The third stage is implemented by `Migrator.ElixirTypeConstructor` module. This component assembles the final result in the mode requested by the user: direct Elixir annotations, direct replacement with expanded user-defined types, internal `Descr` expressions, or compiler-level `Descr` with `@assert_type_form` assertions. It thus serves as the final rendering layer that converts translated types into code that can be written back into a target project from where the `TypeSpec` annotations were extracted. The mode we mainly use in our later experiments is the `Descr` mode with `@assert_type_form` assertions in order to perform type-check on the translated types. Therefore, we substitute `$` used in the specification for signifying annotation for `@assert_type_form` in our experiments for data gathering.

2.1.3 Dataset producer

The dataset producer automates end-to-end analysis across multiple Mix projects. Its pipeline is:

1. *Translation execution*
2. *Dialyzer analysis*
3. *Typechecker execution*

Translation execution is orchestrated by the task defined in `lib/mix/tasks/run_translation.ex`. This task configures logging and delegates to `TranslationRunner.main/1`, which traverses a directory of projects and applies the migrator to each source file.

Dialyzer analysis is performed by `lib/mix/tasks/run_dialyzer.ex`. This task sets up a file logger and invokes `DialyzerRunner.main/1` to run Dialyzer across the Mix projects from which we extracted `TypeSpecs` for translation in the previous stage, collecting warnings and type analysis results for the dataset.

Typechecker execution is performed by `lib/mix/tasks/run_typecheck.ex`. It similarly configures logging and calls `TypecheckRunner.main/1`, optionally with an explicit Elixir binary path (in our case the location of the compiler on which we can use assertions), to run the new compiler's typechecker across the same projects. The outputs of these tasks provide the empirical validation data used in the analysis.

2.2 Dataset Construction

To analyse the translation and to validate the output empirically, we constructed a raw dataset drawn from real world open-source Elixir projects. The dataset records extracted `TypeSpec` annotations, translated Elixir Type annotations, function definition, associated user-defined (or remote) types, and outcomes of Dialyzer and the new compiler (typechecker). This section describes how the dataset is built, how each tool is invoked, and how the results are structured.

2.2.1 Overview

The dataset covers **383 repositories from 54 distinct owners** (GitHub organisations and individual maintainers, with three repositories hosted on other forges) and contains 23 073 **translated function specifications**. Construction proceeds in three sequential stages driven by three Mix tasks:

1. *Translation execution* (`mix run_translation`), which translates every `@spec` in the corpus and emits `dataset.jsonl`.

2. *Dialyzer analysis* (`mix run_dialyzer`), which runs Dialyzer on the translated projects and enriches each dataset entry with Dialyzer outcomes.
3. *Typechecker execution* (`mix run_typecheck`), which runs the new Elixir compiler’s typechecker and enriches entries with its outcomes.

Each stage reads and rewrites the same target dataset file so that the three stages compose into a single artefact.

2.2.2 Dataset Entry Schema

Each line of `dataset.jsonl` is a JSON object with the following fields:

- `project`, `module`, `function`, `arity`: the identity of the function.
- `file`: the source file path relative to the corpus root (e.g. `cldr/cldr47/lib/cldr.ex`). Because the same (`module`, `function`, `arity`) may occur in several sub-projects under one organisation, this field is what disambiguates a warning’s true target.
- `spec`: the original `@spec` annotation as a string.
- `elixir_type`: the translated annotation produced by the migrator.
- `type`: the list of `@type` definitions in scope for the function.
- `definition`: the full function source reconstructed from the AST.
- `spec_lines`, `start_line`, `end_line`: source location metadata.
- `dialyzer`: an object with fields `analysis` ("success" or "failed"), `pass` (Boolean), and `messages`. Each message records its `category`, `line`, `message text`, and an explicit target (the `Module.function/arity` the warning is attributed to).
- `typecheck`: an object with fields `analysis`, `pass`, and `messages`, each message recording `target`, `line`, and `message`.

2.2.3 Stage 1: Translation Execution

Translation is orchestrated by `TranslationRunner.main/1`. It begins with a full-corpus type-cache build: all source files are passed to `Migrator.TypeTranslator.process/1`, which scans every `@type`, `@opaque`, and `@typep` declaration across the repository and builds a module-indexed table of parsed type definitions. This global pre-pass is necessary so that user-defined types are available for substitution when `TypeSpec` annotations from individual files are translated later.

Files are then processed concurrently using `Task.async_stream` with one task per source file and a concurrency limit of `System.schedulers_online()`, matching the number of logical CPU cores available at runtime—eight on the Apple M1 used here, so at most eight files are translated at once. Each task is independent: it reads the source file, extracts specs, invokes the migrator, and produces a list of dataset entries without communicating with other tasks. Each task proceeds as follows.

TypeSpec extraction. The file is parsed into an AST and walked by a recursive extractor that traverses `defmodule`, `__block__`, `@spec`, `def`, and `defp` nodes. For each function that has a matching `@spec`, the extractor records a structured entry containing the spec text, input and output AST nodes, guard clauses, source line numbers, and the full function body reconstructed via `Macro.to_string/1`. End-line positions are derived by walking the function’s AST with `Macro.prewalk` and tracking the maximum line number seen.

Elixir Type injection. The migrator is invoked with the `:descr_assert` output mode on the pre-modification file, producing a list of translated annotations keyed by the `@spec` line numbers. This is done before any file writes so that the line numbers used by the extractor and by the migrator are consistent. The migrator also inserts `@assert_type` form lines on a line above corresponding `TypeSpecs` in the target source file; these are later consumed by the typechecker.

Struct representation and its resolution limit. A struct is translated to a map whose `__struct__` key holds an atom naming the struct’s module. For `%User{}` one might expect the bare `:User`, but an Elixir module name is an `Elixir.`-prefixed atom (Elixir Core Team (2024a)), so the translation writes `%{__struct__: "Elixir.User"}`, not `%{__struct__: :User}`. This prefix is added at the final rendering step—when the translated type is emitted as an Elixir Type, not during the translation itself—so that the `__struct__` atom matches the module’s runtime atom on the BEAM, which is what the typechecker compares the annotation against. When every field of the struct is resolvable, it is emitted as a *closed* map, faithful to the fact that a struct admits no keys beyond its declared fields. When the defining module lies outside the translation path its `defstruct` is unavailable, so the field types cannot be recovered and the closed form cannot be written; the translator instead emits an *open* map that keeps the `__struct__` key and wraps the whole type in `dynamic()`, exactly as it approximates every other unresolvable definition (a remote type outside the path is likewise replaced with `dynamic()`). This broadening is deliberate: an unresolved struct still has fields, but with no way to recover their types the translator chooses a type that is *compatible* with the struct rather than committing to a closed shape that would be wrong. Two residuals, however, still force a *wrong* closed shape, and neither is reducible without a compiler-level lookup table:

- **unexpanded alias** — a struct written through an alias the AST walk cannot resolve (for instance one injected by a `use` macro rather than a literal `alias` directive). The compiler resolves `%Graph{}` to `Scenic.Graph`, but the translator appends the `Elixir.`-prefix to the bare alias, emitting `"Elixir.Graph"` instead of `"Elixir.Scenic.Graph"` — a different module atom.
- **unrecovered field set** — the type cache collects `@type`, `@opaque`, and `@typep` declarations but not `defstruct`, so a struct’s full field set is known only when a complete `@type t` is in scope; otherwise the closed map lists only the fields the annotation itself names.

In either case the emitted closed map is *disjoint* from the real struct type — a wrong `__struct__` atom, or a smaller key set — rather than the sound open-map widening of the previous case. This is the genuine cost of AST-based translation without a persistent lookup table: the compiler holds the alias environment, the `defstruct` field set, and every dependency’s compiled type information, whereas the source-level translator does not. The effect of these disjoint types under the two checkers—and why only some of them surface as warnings—is examined in Subsection 2.3.5.

Annotation matching. Translated annotations are joined to spec entries by matching on the `@spec` line number. Each entry also records its `file`, which later keys warning attribution to the correct definition. The result for each file is a list of entries ready for serialisation.

Fault tolerance. Each file task is wrapped in `with_retry/2`, which re-attempts the task up to three times on exception and logs a warning on each failure. Tasks that exceed a 60-second timeout are killed and omitted from the output.

The entries from all tasks are collected and written to `results/dataset.jsonl` as newline-delimited JSON.

DatasetWriter. For runs that accumulate results incrementally, a `DatasetWriter` GenServer provides concurrent write access to the file. Write requests arrive as asynchronous `cast` messages so that callers are never blocked. Before each write, the server loads the existing file, computes a deduplication key of `(project, module, function, arity)` for each incoming entry, removes any existing entries with matching keys, and then writes the merged content atomically. A synchronous `flush` call, used at shutdown, drains the message queue before the process terminates.

2.2.4 Stage 2: Dialyzer Analysis

Dialyzer analysis is performed by `DialyzerRunner.main/1`. The runner loads the existing dataset to determine which project organisations are present, then discovers Mix project roots under the Dialyzer copy of the corpus by globbing for `mix.exs` files—skipping `deps`, `_build`, and test directories, and keeping only the top-level `mix.exs` of each branch so that nested projects (umbrella sub-applications, vendored dependencies, and example apps) are never counted or analysed separately.

Dependency injection. Before running Dialyzer, the runner checks whether dialyxir is already listed as a dependency in the project's `mix.exs`. If it is absent, the runner injects `{:dialyxir, "~> 1.4", only: [:dev, :test], runtime: false}` into the dependency list using a regular-expression replacement. This allows Dialyzer to be invoked through `mix dialyzer` regardless of the original project configuration.

Project processing. Each project is processed concurrently. The runner invokes `mix deps.get`, `mix compile`, and `mix dialyzer` in sequence using `System.cmd/3`. If dependency resolution or Dialyzer itself fails, the project is marked as skipped and its entries receive `"analysis": "failed"` in the dataset.

Warning extraction. Dialyzer's output is parsed in two passes to handle the two output formats produced by different versions of dialyxir. The first pass detects legacy-format blocks introduced by the string "Legacy warning:" and normalises them. The second pass operates on the structured section beginning at "Total errors:". Within each section, warning blocks are delimited by lines matching the pattern `lib/*.ex:N:` and are parsed with a regular expression that extracts the file path, line number, warning category, and message text. The modern dialyxir format tags each warning with a structured category (`no_return`, `call`, `pattern_match`, and so on); the legacy format does not, so for legacy blocks the category is inferred from the message text and the full message is preserved.

Because Dialyzer reports only a file and line, never the enclosing function, each warning is mapped back to a (module, function, arity) target by re-parsing the source file and locating the spec'd clause whose line range contains the warning line. The warning's file is recorded relative to the corpus root so that it can be matched against the entry's file field.

Dataset enrichment. Every warning must be attributed to the function that actually produced it, since the dataset preparation later keeps or drops each definition according to its own verdicts. After all projects have been processed, the runner iterates through the base entries and, for each entry, finds the Dialyzer warnings whose (file, module, function, arity) match the entry's identity. The file component is essential: the `cldr` organisation alone ships four release versions as separate sub-projects, each defining `Cldr.validate_script/1` and many others, so a key omitting the file would broadcast a warning from one version to every same-named entry. An entry passes if no matching warnings exist. The enriched entries are written back to `dataset.jsonl`.

2.2.5 Stage 3: Typechecker Execution

Typechecker execution is performed by `TypecheckRunner.main/2`. The key difference from the Dialyzer stage is that the new Elixir compiler's typechecker is invoked via a custom Elixir binary built from the development branch of an Elixir version (1.21.0-dev) rather than through a standard Mix plugin, since the stable 1.19.5 releases neither expose the typechecker nor recognise the `@assert_type` form directive. The runner prepends the directory of the custom binary to the shell `PATH` before invoking `mix compile`, so that the typechecking compiler is used in place of the system Elixir installation.

Warning parsing. The typechecker emits warnings in the format `warning: <message>` followed by a location line beginning with a box-drawing corner prefix that records the file path, line number, and a fully qualified `Module.function/arity` identifier. Unlike Dialyzer's line-only output, this location line names the target function authoritatively, so no source re-parsing is needed to attribute a typecheck warning. Three properties of the compiler's output must be handled for the parse to be complete and correct: most warning headers are *indented*, the location line carries an optional column and an occasional (app version) prefix, and the human-readable message is stored separately from the authoritative target.

Arity mismatch. Although Dialyzer allows omitting an optional argument's type, the new compiler does not support such behaviour yet, hence, it mandates including an optional argument's type and fails when a specification does not contain the full argument types, evoking an arity mismatch error. To increase quantity of typechecked entries, we visited every arity failure and added a `dynamic()` type as a filler.

Dataset enrichment. Matched warnings are joined to dataset entries by the (file, module, function, arity) key, following the same file-disambiguated pattern as the Dialyzer stage. The relative file path is invariant between the two copies, so a Dialyzer warning (computed on the Dialyzer copy) and a typecheck warning (computed on the translation copy) both match the single entry whose file they share. The final `dataset.jsonl` contains complete records with both tool outcomes, each carrying its explicit target, for every translated function.

2.3 Dataset Analysis

2.3.1 Hypothesis

The new Elixir typechecker is designed to be more error-prone than Dialyzer. The set-theoretic typechecker performs a structural analysis of each function body against its declared and inferred types and warns on definitions that violate them. On this setting, every function definition Dialyzer warns should also be warned by the typechecker, and the typechecker should additionally find potential type errors Dialyzer cannot detect. Formally:

$$\begin{aligned} \text{Dialyzer fails} &\Rightarrow \text{Typechecker fails} \\ \text{Dialyzer passes} &\Rightarrow \text{Typechecker fails} \vee \text{passes} \end{aligned}$$

We expect the implications to hold together with the strict superset expectation that the typechecker fails on substantially more definitions than Dialyzer.

To judge whether the hypothesis holds we examine the *intersection* of the two tools’ warnings, that is the fraction of both warned by Dialyzer and the typechecker over only Dialyzer-warned:

$$\frac{\text{Dialyzer fails} \wedge \text{Typechecker fails}}{\text{Dialyzer fails}} \approx 1.0$$

The closer the fraction to one confirms the hypothesis: the new system preserves Dialyzer’s discrimination and exceeds it. Where the fraction yields less than one, we then reason about the residual, i.e., the entries Dialyzer warns on but the typechecker does not.

The analysis addresses three questions:

1. To what extent is the typechecker stricter than Dialyzer, and how much of Dialyzer’s discrimination does it subsume?
2. Where the subsumption breaks down, why does Dialyzer catch an error that the typechecker does not, and is that a fundamental disagreement or a limitation of the present state of either the typechecker or the translation?
3. What does the new typechecker detect that Dialyzer does not, and what does this imply for downstream use?

2.3.2 Overall Results and Verdict

The dataset construction described in Section 2.2 produced 23073 translated function specifications. Dialyzer analysed 22982 of them and failed on 91; the new typechecker analysed 22621 and could not analyse the remaining 452, whose host projects fail to compile under the custom compiler (a dependency that either tool rejects aborts the whole project; see Section 2.2). The 91 Dialyzer could not analyse are a subset of those 452 — entries neither tool could analyse — so both tools return a verdict on the same 22621 entries. The cross-comparison below is therefore over those 22621 entries both tools resolved, with the 452 reported separately. Table 2.1 summarises the outcomes:

Two observations stand out. First, the typechecker is decisively the stricter of the two analyses: it rejects 3835 entries (16.6% of the corpus) where Dialyzer rejects only 297 (1.3%), a thirteen-fold difference. The set-theoretic typechecker’s pass rate (81.4%) sits below Dialyzer’s (98.3%). Second, where the two tools fail, they overlap substantially but not completely.

Outcome	Count	% of total
Dialyzer passed	22 685	98.3%
Dialyzer failed	297	1.3%
Dialyzer null	91	0.4%
Typechecker passed	18 786	81.4%
Typechecker failed	3 835	16.6%
Typechecker null	452	2.0%
Both tools passed	18 688	81.0%
Both tools failed	199	0.9%
Both tools null	91	0.4%
Typechecker-only failed	3 636	15.8%
Dialyzer-only failed	98	0.4%

Table 2.1: Tool outcomes across 23 073 translated function specifications. A *null* outcome means the tool could not analyse the entry because its host project failed to compile — for the typechecker, under the custom development compiler. Dialyzer’s 91 nulls are a subset of the typechecker’s 452, the entries neither tool could analyse; percentages are of the 23 073 total.

Subsumption rate on failures. Dialyzer warned on 297 entries in total; the typechecker also warned on 199 of them, giving a subsumption rate on failures of $199/297 \approx 67.0\%$. The remaining 98 Dialyzer-warned entries (the Dialyzer-only set) are the cases where the first implication appears to break down. However, as Subsection 2.3.4 establishes, none of these 98 represents a case in which the typechecker disagrees with Dialyzer; each is attributable to an analysis the typechecker does not perform, to a feature (@opaque-type enforcement) it does not yet model, or to precision lost during approximation.

Verdict. The results support the hypothesis’ subsumption clause but do not qualify its strict-superset reading. The new typechecker does warn on far more definitions than Dialyzer, just as predicted; but, as Subsection 2.3.5 shows, most of that excess is translation residuals — struct patterns the translator could not fully resolve from the AST and positions widened to `dynamic()` — rather than superior error detection. The first implication, that if Dialyzer warns the typechecker warns too, holds directly for 67.0% of Dialyzer’s warnings, and none of the remaining cases is a real disagreement. Of the 98 entries Dialyzer catches but the typechecker does not, each is attributable to a whole-program check the typechecker does not perform, finding it makes locally but that translation masked with `dynamic()`, or an @opaque-abstraction violation the type system does not yet model, none a genuine type-theoretic disagreement (Subsection 2.3.4 gives the breakdown). In short, the two analyses are complementary rather than nested: where both resolve the same types the typechecker reproduces Dialyzer’s findings, but each also flags a class the other cannot, namely Dialyzer the whole-program properties and intra-project calls the typechecker leaves as `dynamic()`, and the typechecker a core of purely-static body-level return leaks that Dialyzer’s optimism accepts (Subsection 2.3.5).

2.3.3 Approximation via `dynamic()`

Out of the 23 073 entries, 13 910 (**60.3%**) contain at least one `dynamic()` sub-expression in the translated annotation. This widening is the single most consequential approximation in the translation, and it is the mechanism behind a large share of the residual disagreement analysed below.

When, for example, the definition of a remote type `Module.type_name()` is not available at translation time (because the module lives in a dependency outside the translation path), the translator conservatively replaces the reference with `dynamic()`. Because `dynamic()` is compatible with every type, the resulting annotation is no stronger than the original specification with respect to its unresolved components: the typechecker, checking a body against an argument typed `dynamic()`, cannot derive a contradiction that Dialyzer derives from the original

concrete contract.

A subtlety of the new system sharpens this picture: `dynamic()` is always hoisted to the root of its enclosing container Elixir Core Team (2024b). When the translation produces a nested gradual type such as `{:ok, dynamic() }`, Elixir rewrites it to `dynamic({:ok, term()})`: the whole tuple becomes gradual, not merely its second element. The widening is thus coarser than its textual position suggests: a single unresolved remote type anywhere inside a structure makes the entire structure gradual, so the masking analysed below operates at the granularity of the enclosing tuple, map, or list rather than the individual leaf. The same property carries an important converse: an annotation containing *no* `dynamic()` at all is checked as a fully static type: “if the user provides their own types, and those types are not `dynamic()`, then Elixir’s type system behaves as a statically typed one” Elixir Core Team (2024b). A `dynamic()`-free annotation is therefore a genuinely static, fully checked specification, which is exactly why the precise training pool (Subsection 2.3.7) and the safe-and-precise criterion require `dynamic()` to be *entirely absent* rather than merely rare. The direct consequence is the one to keep in mind when reading a *pass*: where the annotation carries `dynamic()` the typechecker accepts trivially, so a *pass* certifies only the concrete positions and leaves open whether the static type would pass or warn. For a wholly `dynamic()` annotation, such as the opacity cases of Subsection 2.3.4, a *pass* is no static verdict at all.

The prevalence of `dynamic()` does not invalidate the translation; it quantifies how much of the corpus carries cross-module type references that cannot be refined without expanding the translation scope to the full dependency tree. Its effect runs in both directions and is made precise in the next two subsections: it suppresses some Dialyzer-detectable errors (Subsection 2.3.4), yet 2747 of the 3835 typechecker-rejected entries (72%) still carry a `dynamic()` and are rejected regardless, showing that the widening is far from a blanket suppression of the typechecker.

2.3.4 Dialyzer-Only Warnings: Why Dialyzer Catches What the Typechecker Cannot Yet

The 98 Dialyzer-only entries are the crux of the hypothesis: they are precisely the cases where Dialyzer rejects a definition the typechecker accepts. However, establishing that none of them is a true type-theoretic counterexample is what allows the first implication to stand. Table 2.2 breaks them down by warning category; the category names and their meanings follow Dialyzer’s own classification Ericsson AB (2024a), as surfaced through the `dialyxir` interface used during construction Aronoff and Dialyxir Contributors (2024). A counting caveat first: Dialyzer routinely reports several warnings for a single function definition (these 98 entries carry 134 warnings between them), whereas the new typechecker generally surfaces only one per function, a difference that matters once both tools fire on the same definition (Subsection 2.3.6). The column **N** below therefore counts the *entries* that carry each category, not raw warnings. A representative entry for each category, namely the function and the Dialyzer message it draws, none of which the typechecker reproduces, is given in the appendix “Appendix. Dialyzer-Only Warning Examples”.

Classifying the 98 entries by *why* the typechecker does not reproduce the warning yields a clean three-way partition:

- **59 entries** fall solely in categories the set-theoretic typechecker does not analyse at all: the whole-program reachability and dead code behind `no_return` and `unused_fun`, and the cross-module resolution behind `unknown_function`, `call_to_missing`, and `callback_type_mismatch`.
- **27 entries** carry a finding the typechecker performs *locally* and reproduces elsewhere, namely an argument mismatch (`call`), a dead pattern or clause (`pattern_match`, `pattern_match_cov`), or an impossible guard (`guard_fail`), but missed here; the causes, translation widening the scrutinised type to `dynamic()` versus a genuine residual gap, are examined below.
- the remaining **12 entries** carry *solely* an opacity warning (`call_without_opaque`, `opaque_match`) for `@opaque` type that the typechecker does not model and the translation cannot represent.

None of the three is a case where the typechecker, given the same information and feature set, type-theoretically disagrees with Dialyzer. We treat each in turn.

Whole-program analyses outside its scope (59 entries). By design the new typechecker performs *module type inference*: it infers and checks each function using the current module, the standard library, and dependencies,

Category	N	What Dialyzer reports	In typechecker?
no_return	29	A function never returns normally; it always raises, exits, or loops.	no (whole-program)
pattern_match	18	A pattern can never match the type of the term it is matched against.	partial (local)
call_without_opaque	12	An @opaque-typed argument is supplied by its raw internal structure.	no (opacity)
unused_fun	12	A function is defined but never called anywhere.	no (whole-program)
call_to_missing	11	A call targets a function missing or not exported by its module.	no (cross-module)
unknown_function	9	A call targets a function that does not exist.	no (cross-module)
pattern_match_cov	5	A clause is already fully covered by earlier clauses and is unreachable.	partial (local)
guard_fail	4	A guard test can never succeed for the inferred types.	partial (local)
call	3	A call's arguments cannot match the callee's expected input types, so it can never succeed.	partial (local)
callback_type_mismatch	1	A @behaviour callback implementation's spec conflicts with the behaviour's declared callback type.	no (cross-module)
opaque_match	1	A pattern inspects the internal structure of an @opaque term.	no (opacity)

Table 2.2: Dialyzer warning categories among the 98 Dialyzer-only entries. *N* is the number of entries carrying each category; an entry with warnings in several categories appears in more than one row, so the column sums to more than 98. The final column indicates whether the typechecker performs an analysis that can produce the same finding. *partial (local)* marks a category the typechecker checks inside a function body and therefore reproduces when both tools resolve the same types (Subsection 2.3.6), but misses here because translation widened the relevant type to `dynamic()`; *no* marks a finding outside its analysis altogether, requiring *whole-program* reasoning (call-graph reachability or dead code), *cross-module* resolution, or an *opacity* model it does not have.

while assuming that calls to other modules *within the same project* have type `dynamic()`; its inference is best-effort, warning only on definite failures (cases where *every* combination of a type fails), so a passed function is *not disproven* rather than verified sound Elixir Core Team (2024b). Five Dialyzer categories fall outside this body-local view entirely. `no_return` and `unused_fun` are products of Dialyzer's *whole-program* success-typing fixed point Lindahl and Sagonas (2006): detecting that a function has no reachable return path, or that it is never called, requires reasoning across the program's control flow and call sites, which a current function-at-a-time typechecker does not do. `unknown_function`, `call_to_missing`, and `callback_type_mismatch` require *cross-module* resolution: because the typechecker assumes any call to another module in the same project has type `dynamic()`, it never looks the callee up, and so never notices that the target function is missing or unexported (`unknown_function`, `call_to_missing`) or that an implementation violates the callback type its `@behaviour` declares (`callback_type_mismatch`). On these 59 entries the typechecker found no body-level fault either, so it passed; the warnings are therefore not type disagreements but findings of an analysis it does not *yet* perform, and nothing in its theory precludes adding them. Until then Dialyzer remains complementary here, and the 59 entries are evidence of an analytical-scope difference, not of a definition the typechecker wrongly accepts.

A further consideration discounts the `unknown_function` and `call_to_missing` warnings in particular as trivial. Dialyzer emits one whenever a called function's module is absent from its PLT (the persistent lookup table of analysed modules), so the warning reports *dependency-closure completeness*, not type correctness: the residual cases here call into optional OTP applications (`:eldap`), development and test tooling (IEx, Mix, ExUnit), optional libraries outside the build closure (Jason, PhoenixHTMLHelpers), or compiler-internal and operator-expansion helpers (`:elixir_quote`, `Enum.__in__/2`) that no project declares as an analysed dependency. This sensitivity to the PLT is also why the count is environment-dependent (a handful of these warnings in earlier constructions disappeared once the relevant dependencies were built into the PLT), and it confirms the category is an artefact of the build closure rather than a property of the code. The typechecker, which never resolves cross-project calls in the first place, has no analogue to produce, so these entries cannot be a disagreement.

Locally checkable but unreproduced (27 entries). The remaining non-opacity categories are all within the typechecker's body-local remit: an argument mismatch at a call site (`call`), a pattern or clause that can never match (`pattern_match`, `pattern_match_cov`), and a guard that can never succeed (`guard_fail`) are each a body-against-declaration inconsistency it is built to catch. That it does catch them is visible in the both-warned set (Subsection 2.3.6), where it independently emits the same finding on 36 of 78 `call` entries, 30 of 55 `pattern_match`, and a few of the others. That it did not reproduce these 27 is a matter of approximation

and analysis scope, not of an inability to express the check itself. For only 6 of them could the cause be direct `dynamic()` masking: the annotation still carries a `dynamic()` that can cover the position Dialyzer faults, so where Dialyzer reasons against the original concrete type and finds the call impossible or the pattern dead, the widened annotation presents the typechecker with a value that vacuously satisfies the call and makes every pattern reachable (Subsection 2.3.3); with the type translated concretely the typechecker would be expected to reproduce the finding. The other 21 carry *no* `dynamic()` at all, and their annotations are faithful renderings (a source `map()` becomes `open_map()`, an `any()` becomes `term()`, with nothing widened), so they are a genuine residual gap: real dead-clause, unreachable-pattern, and impossible-guard findings the typechecker simply does not make. Each of the 21 hinges on whole-program information the module-local analysis lacks. The unreachability follows from the return type of an intra-project call (a same-project callee returns an error tuple where the code expects a map, as in `as_to_model_data/1`'s `extract_actors/1` and the Mobilizon and Akkoma federation fetchers, which module-local inference assumes is `dynamic()`); from caller-side success typing that narrows an argument below its declared type (a four-atom tag only ever `:hour` or `:week` in `ttl_to_seconds/1`); or from a compile-time module attribute the typechecker does not constant-fold (`if @env == :test`, statically dead in `task_children/0` and `maybe_inserted_at/0`). All need whole-program success typing or compile-time constant-folding that the module-local, best-effort typechecker does not perform: the Dialyzer categories are ones the typechecker can express, but these particular instances require information beyond its scope. They are the same whole-program reach that yields the 59 above, surfacing here under a locally-checkable category rather than as a translation artefact.

Opacity warnings the typechecker does not model (12 entries). The remaining entries carry `opaque_match` or `call_without_opaque`: Dialyzer warns that code constructs or pattern-matches an `@opaque` type through its internal structure rather than its public interface, an abstraction-boundary violation, here on library types such as `Ecto.Multi` and `Mint.HTTP.t()`, and even a project-local opaque (`Concentrate.TripDescriptor`). This is not a type error in the ordinary sense but an opacity-enforcement finding, and the new type system does not distinguish an opaque type from its underlying structure: it does not model the `@opaque` (including `@typep`) type at all. The translation cannot supply what the target system lacks: an opaque type is bluntly translated to its structure, or, when its defining dependency is unavailable, to `dynamic()`, hence the boundary of type security levels is erased on both sides. Empirically the widening applies to eleven of the twelve: they carry a `dynamic()` in the translated annotation and none names the opaque type at all. Crucially, and unlike the locally-checkable group, this is *not* a precision loss that resolving the dependency would repair: even with the opaque type fully available, the typechecker would still not flag the violation, because it has no notion of opacity to violate. Tellingly, the one entry whose annotation contains no `dynamic()` at all (`ExCubicIngestion.ProcessIngestion.ingest/1`, on `Ecto.Multi`) is missed all the same—confirming that opacity, not approximation, is the operative cause.

Project distribution. The Dialyzer-only warnings are concentrated: Mobilizon contributes 50 of the 98 and Akkoma 20, with the remaining 28 spread across twelve further projects (MBTA seven; Analytics five; Absinthe and Cachex three each; `cldr` and `Livebook` two each; and six projects with one apiece). The concentration tracks `dynamic()` density: the projects with the most unresolved remote types are those whose Dialyzer findings the translation most often masks with the gradual type.

2.3.5 Typechecker-Only Warnings

3636 entries triggered a warning from the typechecker that Dialyzer did not report. At face value this is the strict-superset clause of the hypothesis; on inspection, however, most of it is not superior error detection but translation residuals and the conservatism of module-local inference. The findings divide by kind into unreachable clauses or patterns (1677), argument mismatches at a call site (914), return-type mismatches (691), map-key accesses (217), and a residue of other checks (137).

Unreachable-clause findings (1677). The largest group is not the typechecker over-warning but the typechecker *correctly* rejecting a wrong translated type: 1308 of the 1677 reject a struct pattern against a translated struct type that is *disjoint* from the real struct, every one a residue of translating structs from the AST without a symbol table (Subsection 2.2.3). In 752 the translated struct type lists only some of the struct's fields, so the closed map

excludes valid structs and a `%Mod{}` pattern cannot match; in 556 the translator could not expand an aliased struct name to its full module path, so the asserted `__struct__` atom names a different module than the pattern's. A verified instance is `Scenic.Components.button/3`, whose clause head pattern-matches the struct:

```
65 def button(%Graph{} = g, data, options) do
66   add_to_graph(g, Component.Button, data, options)
67 end
```

The compiler resolves `%Graph{}` to `%Scenic.Graph{}` (`: "Elixir.Scenic.Graph"`), disjoint from the translated argument's `: "Elixir.Graph"`, so the typechecker reports the clause unreachable; Dialyzer, reading the original spec, accepts the same clause, confirming the code is correct and the fault is the translation. The warning is thus *valid* — a sound deduction from the type it was handed — yet it exposes a translation defect rather than a program fault, so it is neither a type-theoretic disagreement nor genuine error detection; both the field-set and alias residuals would be removed by resolving structs against their defining modules. Whether the defect surfaces at all, however, depends on the body: only a struct the body *exercises*—through a pattern or a field access—is caught. The same `: "Elixir.Graph"` defect in `Scenic.Primitives.rrect/3`, which merely forwards its argument, produces no warning and is admitted into the both-pass pool, sound but unfaithful to the specification:

```
68 def rrect(graph_or_primitive, rrect, opts \ [] ) do
69   rounded_rectangle(graph_or_primitive, rrect, opts)
70 end
```

The 1308 counted here are therefore only the *exercised* residuals; an unknown further number pass silently into the pool. The remaining 369 are non-struct, and they include the genuine dead code the typechecker surfaces and Dialyzer accepts: a clause matching an atom outside the scrutinee's type (`: dev_blue` where the value is only `: dev_green` or `: prod`), a defensive fall-through made redundant by an exhaustive earlier match, a right-hand `||` branch that can never run because the left side is always truthy, or a constantly-true-or-false guard. For a codebase moving off Dialyzer onto the new checker, this reachability and exhaustiveness feedback is precisely the kind of static signal the transition is meant to add.

Argument mismatches (914). These are the conservatism of module-local inference: the faulted argument is inferred `term()` or `dynamic()`, from an intra-project call or a broad declared type such as `map()` or `list()`, so the typechecker cannot establish the callee's expected input and warns on correct code; others pass a struct whose translated type dropped the field the callee reads, turn on protocol dispatch (for instance `Ecto.Queryable`) the checker does not model, or are spec-looseness, where a bare `map()` parameter becomes an open map that guarantees no field, so a field access in the body cannot be shown to succeed even though the runtime map carries the key.

Return-type findings (691). These classify by the typechecker's inferred body type against the declared type into three groups. In 390 the annotation itself carries a `dynamic()`: after root-hoisting it is gradual, so the disagreement is on a non-static type and neither a pass nor a warning here is a static verdict (Subsection 2.3.3); this group absorbs the translation artefacts, a concrete body warned against a `dynamic()` assertion that root-hoisting stripped of its optionality, or over-broadened inference that invents a union arm the body never returns. In a further 125 the inferred return type is `dynamic()` or the top type `term()` while the asserted type is concrete: the body returns the result of an intra-project or un-spec'd remote call that inference assumes is `dynamic()`, so the typechecker cannot establish the concrete return type and warns, whereas Dialyzer's whole-program success typing resolves the callee and confirms the contract (`def config_path, do: Path.join(data_dir(), ...)` is inferred `dynamic()` against a declared `binary()` because `data_dir/0` is intra-project). The remaining 176 are purely static, no `dynamic()` on either side, so a disagreement is a true static verdict: the body provably returns a value outside the declared type that Dialyzer's optimism accepts, as when `Scenic.PubSub.publish/2` returns `{:error, :not_registered}` under the specification `:ok` or `Absinthe.Type.Custom.parse_date/1` returns `:error` under `{:ok, nil}`. These purely-static leaks are the legitimate content of the strict-superset clause; representative cases are given in the appendix "Appendix. Typechecker-Only Warning Examples".

Interpretation. The headline "3636 warnings Dialyzer misses" overstates the typechecker's advantage — not because the typechecker is wrong (its warnings are valid) but because most are correct responses to a faulty *input*,

the translated type, rather than detections of a faulty *program*. The unreachable-clause group is dominated by struct-representation residuals of AST-based translation (1308 of 1677), where the checker rejects a disjoint struct type, and the return-type group by `dynamic()`-masked and cannot-verify cases that carry no static verdict (515 of 691). What the typechecker alone genuinely catches is the 176 purely-static return leaks Dialyzer’s optimism accepts and the 369 non-struct unreachable-clause findings its reachability checking surfaces. The two analyses are thus better described as *complementary* than nested for now: Dialyzer flags the whole-program properties and intra-project calls the typechecker leaves as `dynamic()`, and the typechecker both the return leaks Dialyzer’s optimism accepts and the dead code its reachability checking surfaces. As the migration target shifts from Dialyzer to the new checker, these two classes, the return leaks and the reachability findings, are what the codebase stands to gain.

2.3.6 Entries Flagged by Both Tools

199 entries triggered warnings from both tools, the core of the subsumption rate. Their Dialyzer categories are led by `call` (78), `no_return` (70), `pattern_match` (55), and `call_without_opaque` (24), a mix of the locally-checkable and whole-program categories of Subsection 2.3.4, but here arising in functions whose bodies *also* violate their declared return or argument types. These are the unambiguous errors: a definition wrong enough that Dialyzer’s success typing and the typechecker’s body check independently reject it. The typechecker reports a single warning per function (158 of the 199 carry exactly one), and which one it is depends on the category: for the locally-checkable categories it reproduces Dialyzer’s finding directly (incompatible arguments on 36 of 78 `call` entries, a dead pattern on 30 of 55 `pattern_match`), whereas for the whole-program categories it cannot, and rejects instead on a coexisting body-level fault; across the 35 entries carrying only whole-program or cross-module Dialyzer categories, that fault is an unreachable clause in 21, a map-key access in 6, an argument mismatch in 5, a return-type mismatch in 2, and another check in 1. Either way the entry is co-rejected: the typechecker need not match Dialyzer’s diagnosis, only fail the function on some ground. Their existence confirms that the two analyses agree wherever a definition’s fault is visible to both, and that the disagreement studied above is confined to the analytical-scope and approximation boundaries, not to the verdict on plainly faulty code. One entry per Dialyzer category from this set, paired with both tools’ messages, is given in the appendix “Appendix. Both-Warned Examples”.

2.3.7 Implications for Downstream Use

The two-tool cross-validation is not only a quality check on the translation; it directly shapes how the resulting dataset is consumed in the LLM training pipeline of the next chapter, whose goal is a model that predicts annotations the typechecker accepts, that is, predicts *safe*. Three observations transfer directly.

Keeping only typechecker-verified labels. The model is trained to produce annotations the typechecker accepts, so every training label must itself be an annotation the typechecker admits. We therefore keep only the *both-pass* entries, those accepted by *both* Dialyzer and the new typechecker. This is a substantive filter, not a cosmetic one: it removes 4385 entries (19.0% of the corpus), the great majority typechecker rejections, plus the 452 the typechecker could not analyse and the 297 Dialyzer warnings. About a fifth of the translated annotations are thus rejected by, or unverifiable under, at least one tool (genuinely inconsistent for some, but for most only unverifiable by the typechecker’s conservative inference; see Subsection 2.3.5), and excluding them leaves only labels the typechecker positively accepts.

Two datasets: with and without gradual types. Within the both-pass pool one choice remains: whether to also drop entries whose annotation contains `dynamic()`. Subsection 2.3.3 showed this `dynamic()` is mostly an artefact of unresolved remote types, not a deliberate gradual-typing decision, and the choice has costs on both sides: dropping it yields purely static, precise labels but discards more than half the pool, while keeping it preserves coverage at the risk of teaching the model to emit `dynamic()` to avoid type errors. Rather than decide in advance, we build *two* datasets and let the next chapter settle the matter empirically:

- the full **both-pass** pool (18688 entries), which *includes* gradual (`dynamic()`) types; and
- its **dynamic-free** subset (7813 entries) of *purely static* types only.

The two are nested (the second is the precise core of the first); a length filter and a subproject-stratified split (Subsection 3.1.2) give the final per-track counts. Training one model on each lets us test directly whether including gradual types helps or hurts safe, precise prediction.

Typechecker as the success criterion. Because the model is optimised to satisfy the typechecker, evaluation uses the typechecker as ground truth: a predicted annotation counts as correct when the typechecker accepts it on injection into the real project. Dialyzer plays no part in scoring predictions: its role was confined to building the dataset (the agreement study of this chapter and the both-pass filter). The prediction metric is the typechecker’s acceptance alone, which is appropriate because the task is precisely to produce annotations the new type system admits.

2.3.8 Summary

- **Coverage:** all 23073 specs from 383 repositories were translated; Dialyzer analysed every one, and the new typechecker all but 452 whose host projects fail to compile under the development-branch compiler, leaving 22621 with a verdict from both tools.
- **Relative strictness:** the typechecker rejects 3835 entries (16.6%) against Dialyzer’s 297 (1.3%); pass rates are 81.4% versus 98.3%. The typechecker is decisively the stricter tool, as hypothesised.
- **Subsumption:** the typechecker independently warns on 67.0% of Dialyzer’s failing entries (199 of 297), reproducing Dialyzer’s finding directly for the locally-checkable categories and co-rejecting the rest on a separate body-level fault. The 98 it does not co-warn are *not* type-theoretic counterexamples, but split between whole-program analyses it does not perform, locally-checkable findings masked by `dynamic()`, and `@opaque` type violations it does not model (Subsection 2.3.4).
- **Approximation cost:** 60.3% of entries contain `dynamic()`; it suppresses some Dialyzer-visible errors but leaves 72% of typechecker rejections intact, so it is not a blanket suppression.
- **Typechecker-only (15.8%):** its dominant kind is unreachable-clause findings, of which 1308 of 1677 reject struct patterns the translator could not fully resolve from the AST (incomplete field sets or unresolved module paths) rather than genuine errors; the return-type group is likewise dominated by `dynamic()`-masked and cannot-verify cases carrying no static verdict. What it alone genuinely catches is 176 purely-static return leaks Dialyzer’s optimism accepts and 369 non-struct dead-clause findings its reachability checking surfaces (Subsection 2.3.5).
- **Both tools (0.9%):** definitions faulty enough that both analyses independently reject them, confirming agreement wherever a fault is visible to both.
- **Training pools:** the both-pass filter yields 18688 typechecker-verified entries; its `dynamic`-free subset (7813) and the full set define the two supervision tracks compared empirically in the next chapter.

The hypothesis is supported in its subsumption clause but qualified in its strict-superset clause. The set-theoretic typechecker subsumes Dialyzer wherever both reason over the same resolvable types. Its residual blind spots on Dialyzer’s failures are attributable to whole-program analyses it has not *yet* implemented, to features such as `@opaque`-type enforcement it does not yet model, and to translation-time approximation, not to any case where it wrongly accepts a definition Dialyzer soundly rejects. It also warns far more often than Dialyzer, but that excess is mostly translation residuals — struct patterns it could not fully resolve from the AST and positions widened to `dynamic()` — rather than superior error detection, with a genuine core of 176 purely-static return leaks and 369 non-struct dead-clause findings that it alone catches. The two analyses are therefore best read as complementary: each detects a class the other cannot.

Part II

Inferring Elixir Types with Large Language Models

Chapter 3

Type Prediction with LLMs

The translation pipeline of Chapter 1 converts existing TypeSpec annotations into Elixir Types, and the analysis of Chapter 2 characterises how the 23073 translated annotations are analysed under the two checkers: 18688 (81.0%) are accepted by both, while the rest are dominated by translation-time `dynamic()` approximation and the typechecker’s conservative module-local inference, with only a small residue of genuine type errors. Yet translation only reaches functions that *already* carry TypeSpecs. Most production Elixir code carries none: in the application code of our corpus, only 15694 `@spec` annotations accompany 119901 `def` and `defp` clauses, so the large majority of functions go unspec’d. Developers omit the optional `@spec` directives; Dialyzer still analyses these functions by inferring success typings Lindahl and Sagonas (2006), but with no declared contract there is no developer’s intended type to hold the body to. A complete migration to the typed environment is therefore not addressed by translation alone; the remaining, dominant, fraction of the codebase requires that an annotation be *inferred*.

This chapter investigates whether a large language model fine-tuned on the corpus of translated, typechecker-accepted annotations can supply such inferences with enough fidelity to drive the migration. The supervised signal is the by-product of the agreement study (Subsection 2.3.7): the 18688 *both-pass* entries that both tools accept, which after a length filter form the training corpus detailed in Subsection 3.1.2. Each entry pairs a function definition with an Elixir Type annotation the typechecker accepts as consistent with that definition, and the supervised task is to recover that annotation from the definition. We pursue two model families in parallel on this both-pass pool—a large decoder-only model and a much smaller encoder–decoder, fine-tuned on the same annotations—and compare how well each architecture supports the prediction task.

3.1 LLM Type Prediction

Having established a corpus of typechecker-accepted annotations, we now ask whether a language model can *produce* such annotations from a function’s source alone. Because the target is a set-theoretic type, we evaluate a prediction not by textual similarity to the reference but by whether it is a type the reference does not contradict, using the type system’s own compatibility judgement (Subsection 3.1.4).

3.1.1 Task Formulation

We frame type prediction as a *conditional text-generation* task: given the definition of a function (i.e., its source text together with any user-defined types in its module scope) the model writes out the corresponding Elixir Type annotation one token at a time, much as a language model produces a sentence. It is *conditional* because what the model writes depends on the input code, and *generation* because the annotation is produced as text rather than selected from a fixed set of labels, which is necessary here because the new Elixir type system is open-ended (arbitrary unions, tuples, and maps with explicit fields) and forms no finite vocabulary to classify into.

Each training entry consists of:

- the qualified module name in which the function is defined;
- the list of `@type`, `@opaque`, and `@typep` declarations in scope at the function;
- the function definition reconstructed from the AST, including all clauses;

- two *grounding* blocks extracted from that definition, which surface the evidence the code itself provides for the type: the *argument patterns* (each clause’s parameter patterns and guards, e.g. `{%User{ }, _} when is_binary(x)`) and the *return expressions* (the outermost constructor of each clause’s tail position, e.g. `{:ok, _}` or, for a value produced by an opaque call, `call: fetch/1`);
- the target: the translated annotation, already accepted by the typechecker.

These fields are assembled into a structured prompt with delimiter sections (`### Module:`, `### Types in scope:`, `### Definition:`, `### Argument patterns:`, `### Return expressions:`, `### Elixir type:`), with the target annotation placed last. The two grounding blocks make explicit, at the input, exactly the code evidence a faithful annotation should rest on: a type is expected only where a pattern, guard, or tail constructor supports it. Training is plain next-token prediction over this whole text (the standard *autoregressive* objective): at each position the model predicts the next token from all the tokens before it, so that, given the prompt up to `### Elixir type:`, it learns to continue with the correct annotation.

This formulation differs from prior work in three respects. First, the target type system is set-theoretic rather than nominal: predictions must produce unions, intersections, and constructor-shaped types with explicit field information, not class names. Second, the task is type *generation* rather than the type *correction* studied by Mengesha (2026). There is no observed (potentially incorrect) type in the input, only the function definition; the model must produce the annotation from scratch. Third, the language is Elixir on the BEAM, an under-represented target in code-LLM training corpora compared with Python or TypeScript, on which most existing type-prediction work has been conducted.

3.1.2 Dataset Preparation

We train on the *both-pass* pool: every entry on which both Dialyzer and the set-theoretic typechecker accept the translated annotation. This is the largest checker-accepted pool, and it admits annotations whose translation contains `dynamic()`—overwhelmingly an artefact of unresolved remote types rather than a deliberate gradual-typing choice (Subsection 2.3.3), but a type the checker nonetheless accepts as safe. A length filter is applied before the split: entries whose function definition exceeds 2,000 characters or whose target annotation exceeds 1,000 characters are removed, as these pathological cases (e.g. `Regex.replace/4`, whose translation expands to over 250 KiB¹) would dominate batch composition without proportionate learning signal.

Subproject-stratified split. The dataset is partitioned by a project-stratified split: evaluating on functions from a project that also appears in training would over-estimate generalisation. The `project` field denotes a GitHub organisation, not a codebase (an organisation such as `mbta` hosts more than a hundred independent repositories), so we split by *subproject*, defined as `project/module_root` where `module_root` is the top-level segment of the Elixir module path (e.g. `mbta/AlertProcessor`). The split assignment is a deterministic hash of each subproject name (a salted MD5 reduced modulo 100), rather than a shuffle of the universe, so that a subproject keeps its assignment across dataset regenerations at a 70/15/15 target ratio: the test set is stable even as the corpus grows, which is what makes results comparable across training rounds. No subproject leaks across train, validation, and test. This yields the splits in Table 3.1.

Split	Entries	Subprojects
Train	9888	279
Validation	3484	60
Test	2127	64

Table 3.1: The canonical both-pass dataset under the salt-42 subproject split.

Of the 2127 held-out test references, 1639 (77.1%) are gradual (their deterministic translation resorted to `dynamic()`, Subsection 2.3.3); the remaining 488 are fully concrete.

¹Its third argument is typed `(... -> String.t())`, a variable-arity function whose `non-term()` return type the translation expands into a union over every fixed arity the BEAM permits, from 0 to 255 (Chapter 1); a 616-character definition thereby yields an annotation of over 250 KiB.

3.1.3 Model and Training

Base model. We fine-tune Qwen2.5-Coder-7B Hui et al. (2024) (the base variant, not instruct) as the foundation model. This choice reflects three considerations. First, the Qwen2.5-Coder technical report Hui et al. (2024) reports that this model matches or exceeds DeepSeek-Coder-7B-Base Guo et al. (2024) and StarCoder2-7B Lozhkov et al. (2024) across standard code benchmarks while remaining single-GPU deployable. Second, Qwen2.5-Coder is pre-trained on over 5.5 T tokens spanning 92 programming languages, including Elixir, a coverage criterion that excludes some alternatives whose Elixir presence is undocumented. Third, the base variant (rather than the instruction-tuned variant) is more suitable for structured completion tasks: we do not need conversational scaffolding, and starting from a base model avoids fighting habits introduced by chat-format training.

Parameter-efficient fine-tuning. At 7 B parameters the model cannot be fully fine-tuned on a single GPU within reasonable time, and the supervised dataset is too small to benefit from full fine-tuning regardless. We therefore apply QLoRA Dettmers et al. (2023): the base model is loaded in 4-bit NormalFloat (NF4) quantisation with double-quantisation enabled, and a Low-Rank Adaptation (LoRA) Hu et al. (2022) layer is attached to all seven linear sub-modules of every transformer block (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, `down_proj`). The LoRA rank is 32 with $\alpha = 64$ and a dropout of 0.05, giving roughly 80 M trainable parameters out of the 7 B base.

Training configuration. Training uses the HuggingFace TRL SFTTrainer with sequence packing at a maximum length of 1,024 tokens. The effective batch size is 16 (per-device batch size 2 with 8 gradient accumulation steps). The learning rate is 2×10^{-4} with cosine decay and a 5% warmup ratio, weight decay 0.01, and bfloat16 mixed precision. The optimiser is paged AdamW with 8-bit state, and the best checkpoint by validation loss is retained. Training runs for two epochs on a single GPU; eval-time batch size is reduced to 1 to avoid out-of-memory during the cross-entropy over the 152,064-token vocabulary.

Compute environment. All training runs were carried out on the LIP6 Convergence cluster, where each job was allocated through Slurm a single NVIDIA A100 GPU, 8 CPU cores, and host RAM. Most runs used a half-GPU MIG slice (the `a100_3g.40gb` profile, 40 GB) rather than a full A100; the 4-bit QLoRA configuration keeps the 7 B base model together with its LoRA adapters and optimiser state comfortably within the 40 GB budget, so the full `a100_7g.80gb` profile was needed only occasionally, and no run approached its time or memory limit. The compatibility evaluation, which requires the compiler with the typechecker rather than a GPU, was run separately on a local workstation.

Training behaviour. The model converges cleanly with no instability. The validation loss bottoms early and drifts up thereafter, so best-checkpoint-by-validation-loss selection retains the optimum rather than the final weights.

Reproducibility. All training artefacts (configuration, prompt template, every dataset split, and the final adapter weights) are versioned with the seeds used at each stage (salt 42 for the hash split, seed 42 for the trainers).

3.1.4 Evaluation: Set-Theoretic Compatibility

A trained model is evaluated by comparing each predicted annotation against the accepted reference annotation on held-out test entries. The question a metric must answer is not whether the prediction is textually identical to the reference, but whether it is a *safe* type for the function: one the reference does not contradict. This is exactly the judgement the set-theoretic type system already provides, so rather than adopt an external similarity heuristic we evaluate predictions with the typechecker’s own type algebra (the `Descr` module, Chapter 1).

Why not exact match. The most direct metric is exact-match accuracy: the fraction of test entries on which the predicted annotation string equals the reference string. This is uninformatively strict for set-theoretic types. Two annotations that describe the identical set of values, for instance a union written as `A or B` versus `B or A`, or a map written with one field ordering versus another, are semantically equal but textually distinct; conversely

two annotations may differ in a single atom yet denote entirely different value sets. A binary string comparison conflates these cases, and, more importantly, it penalises a prediction that is a *sound* answer merely for being less precise than the reference, which is the wrong signal for a migration setting where a safe over-approximation is a success, not a failure.

Compatibility. We therefore adopt *compatibility*, computed by the `Descr.compatible?/2` relation: the gradual-typing consistency judgement of the set-theoretic system itself. A predicted type is compatible with the reference when, treating `dynamic()` as a wildcard that may stand for any type, the two can be reconciled: the static (non-gradual) content of the prediction must lie within the type the reference permits, so the prediction never *asserts* a shape the reference forbids. A compatible prediction is thus a safe annotation for the function. It may be exactly the reference or strictly more general, the limiting case being `dynamic()` itself, which is compatible with every reference; both count as compatible, whereas a prediction that commits to a concrete shape the reference excludes does not. This is the appropriate success criterion for a migration setting: an annotation is acceptable precisely when it does not conflict with the function’s true type, even at some cost in precision.

Computation. Each prediction and reference is parsed into a `Descr` value and the two are submitted to `compatible?/2`; parsing collapses whitespace and union-arm ordering and rewrites any `|` to `or` (the model occasionally emits the `TypeSpec` union symbol out of pretraining habit, though the target system writes unions only as `or`). A prediction that fails to parse is counted as not compatible (reported separately as *null*).

Exact match. Alongside compatibility we report the *exact-match* rate: the fraction of predictions that denote the identical type to the reference, decided by the same type algebra so that a difference of union-arm ordering or map-field ordering does not count against a prediction. Exact match is strictly stronger than compatibility—it additionally requires equal *precision*, not merely safety—and we use it only as a precision indicator, never as the primary criterion, for the reason given above.

Results. The model is evaluated on the common 2127-entry test set with greedy decoding under a grammar constraint that restricts output to well-formed set-theoretic types. It attains 40.4% compatibility (860 of 2127), an exact-match rate of 8.6%, and a null rate of 0.4%. The remainder of this section asks how far this result depends on the model’s architecture and scale, by comparing against a much smaller encoder–decoder.

3.1.5 Architecture Comparison: A Smaller Encoder–Decoder

The canonical model is a QLoRA adaptation of a 7 B decoder-only base. To ask how much of the result is owed to that architecture and scale rather than to the task and data, we compare against CodeT5+-770M Wang et al. (2023), an encoder–decoder model an order of magnitude smaller, which we *fully* fine-tune rather than adapt. Both models train on the same canonical both-pass dataset with the same prompt and are scored on the identical 2127-entry test set, so any difference is attributable to the model, not the data. One asymmetry must be stated: the Qwen decoding is grammar-constrained to well-formed set-theoretic types, whereas the CodeT5+ decoding is *unconstrained*, because the constrained-decoding library does not support its tokenizer; we therefore report each model’s null rate.

Model	Compatible	Exact match	Null
Qwen2.5-Coder-7B (QLoRA, constrained)	40.4%	8.6%	0.4%
CodeT5+-770M (full FT, unconstrained)	52.5%	30.1%	4.2%

Table 3.2: Architecture comparison on the common 2127-entry test set; both models trained on the canonical both-pass dataset with the same prompt.

The smaller encoder–decoder is the stronger model on both axes. It is more compatible (52.5% versus 40.4%, a 12.0-point margin) and far more precise, matching the reference exactly on 30.1% of entries against 8.6% for Qwen—3.5 times as many—while decoding without the grammar constraint and at roughly a tenth of the

parameters. The one cost of the missing constraint appears in the null column (4.2% against 0.4%), but it does not close the gap: crediting all 89 null predictions as compatible would raise CodeT5+ to 56.6%, still above Qwen. Full fine-tuning of a task-matched encoder–decoder thus outperforms parameter-efficient adaptation of a much larger decoder-only model on this task.

3.1.6 Qualitative Analysis

Inspecting individual predictions makes the behaviour behind the aggregate numbers concrete. All examples below are drawn verbatim from the models’ outputs on the common test set.

Exact matches a string comparison would miss. Some predictions differ textually from the reference yet denote the identical type, which the type algebra credits as an exact match where a naive string comparison would not. `Membrane.RTP.Packet.PayloadType.dynamic?/1` has reference `(integer() or integer() -> :true or :false)`; the canonical model predicts `(integer() -> :true or :false)`, collapsing the redundant union `integer() or integer()` to the identical `integer()`. The two denote the same value set, so the prediction counts as exact, and it typechecks.

Recovering precision the translator lost. On targets whose *reference* is gradual, the model frequently supplies a precise type the deterministic translation could not. `AshStateMachine.Charts.mermaid_flowchart/1` has reference `(dynamic() -> dynamic())` — the translation widened both positions to `dynamic()` because their types came from unresolved remote types — yet the model predicts `(atom() -> binary())`, which typechecks. The prediction is *more* precise than the reference it is scored against: the model has reconstructed information the translation discarded, which is why it remains compatible on much of the gradual-reference subset where an exact match against the reference is by definition impossible.

Where the larger model fails and the smaller succeeds. The architecture comparison of Subsection 3.1.5 shows up in individual predictions. `Postgrex.ReplicationConnection.encode_lsn/1` has reference `(integer() -> {:ok, binary()} or :error)`. Qwen collapses the two result arms into a single ill-formed tuple `{:ok or :error, dynamic()}` with a spurious `empty_list()` alternative, a shape the function never returns, and the typechecker rejects it; CodeT5+ reproduces the reference exactly. The same precision gap appears on return structure: `Integer.extended_gcd/2` has reference `(integer(), integer() -> {integer(), integer(), integer()})`, and Qwen discards the tuple’s element structure, collapsing the return to a bare `tuple()` that admits tuples the function never returns and is therefore rejected, whereas CodeT5+ reproduces the three-integer tuple exactly. In both cases Qwen misjudges the precision the code actually determines — overcommitting to a concrete shape the body never produces (`encode_lsn`), or discarding structure the body fixes (the `extended_gcd` tuple) — and a single mis-typed position is enough for the typechecker to reject the whole annotation. Such precision errors are concentrated on functions with non-obvious control flow or opaque struct constructors, exactly the code where the ground-truth shape is hardest to read off the source alone, and they are the dominant source of incompatible predictions for both models. The smaller encoder–decoder is consistently less prone to it. We are deliberately cautious about the cause: CodeT5+ and Qwen differ in more than their encoder — also in fine-tuning regime (full versus low-rank adaptation), pretraining corpus and objective, and parameter count — so this single comparison cannot attribute the gap to any one factor. One plausible contributor is architectural: an encoder–decoder reads the whole function definition bidirectionally before the decoder fixes any output position, a fuller view of the code’s evidence than a strictly left-to-right, decoder-only pass affords, and CodeT5+ is pretrained end-to-end for exactly this kind of understanding-then-generation task Wang et al. (2023). Isolating the encoder’s contribution — for instance by fully fine-tuning a decoder-only model of comparable scale under an otherwise identical regime — is left to future work.

Conclusion

Conclusion

Conclusion and Future Work

This thesis set out to support the migration of existing Elixir codebases to the language’s new set-theoretic type system along the two fronts on which type information is available: translating the types that are already declared as Erlang-style TypeSpecs, and inferring those that are not. Chapter 1 formalised the translation as a set of equations from TypeSpec syntax to Elixir Types, isolating the constructs that cannot be translated precisely and approximating them with the gradual type `dynamic()`. Chapter 2 realised that translation as a prototype, applied it across 383 repositories (drawn from 54 distinct owners) to produce 23 073 translated annotations, and cross-validated each against both Dialyzer and the new typechecker.

The agreement study established that the new typechecker is the substantially stricter of the two analyses, rejecting 6011 annotations against Dialyzer’s 285, while subsuming the majority of Dialyzer’s own findings (a subsumption rate of 60.7%). Where both tools resolve the same types the typechecker reproduces Dialyzer’s warnings; the residual cases are attributable not to disagreement but to whole-program analyses the typechecker does not yet perform, to `@opaque`-enforcement it does not model, and to translation-time `dynamic()` approximation, so the two analyses are complementary rather than nested. The cross-validation yielded 7498 annotations accepted by both tools; these constitute the checker-verified supervision signal for the prediction task.

The prediction study fine-tuned a code LLM to recover these annotations and, through a two-track design, asked whether gradual types should be admitted into the supervision signal. The result is a trade-off rather than a single best choice. A model trained on the precise, dynamic-free pool and one trained on the larger gradual-inclusive pool invert across two equally legitimate notions of success: the gradual-inclusive model recovers more typechecker-accepted annotations (raw pass rate 75.0% versus 63.7%), which serves a *low-noise* migration whose immediate aim is to make a codebase compile; the precise model produces more annotations that are accepted *and* genuinely static (safe-and-precise rate 63.7% versus 29.7%) and never resorts to the gradual type, which serves type safety and downstream usefulness. Critically, the gradual-inclusive model confirms the concern that motivated this work, namely that a model can learn to over-emit `dynamic()` to mute type errors, lifting the bare pass rate while halving precision; it does so despite training on more than twice the data. The practical guidance is therefore a tunable trade-off rather than a single recommended model: admit gradual types when a low-noise transition is the priority, and exclude them when precision is.

Taken together, the translation prototype and the two prediction models cover the two sources of type information in a codebase, and the checker-verified, two-track methodology offers a reusable template for building and evaluating type-annotation datasets for a young set-theoretic type system. What remains is to compose the two methods into a single end-to-end migration of a complete project and to measure it directly, which we set out as the principal item of future work below.

Limitations and Future Work

End-to-end total migration. This thesis defines *total migration* as the state in which every function in a project carries a typechecker-accepted Elixir Type, and it establishes the two enablers separately: a translation prototype that converts existing `@specs` (Chapter 2), and an LLM that predicts annotations for the unspec’d majority (the two-track prediction study). Composing the two into an end-to-end migration of a complete, previously-unannotated project is the natural next step and is left to future work. Concretely, such a study would run the full pipeline, namely target discovery of untyped functions, annotation *inference* (translation where a spec exists and prediction where it does not), *injection* of the resulting annotation into the source, and recompilation in the original project context, and would report a *migration rate*: the fraction of previously-untyped functions that become

compile-clean and free of type warnings. A case study over a handful of projects, with a per-function breakdown of outcomes (clean compile, type warning, or compile failure from a syntactic or unresolved-reference error), would connect the compatibility metric of Subsection 3.1.4 with extrinsic migration success, testing directly whether compatibility with a reference predicts whether a prediction actually migrates.

Evaluation scope and dataset. Several limitations bound the present results and motivate the study above. The extrinsic evaluation is conducted on *held-out* projects rather than within a single evolving codebase. The corpus itself is bounded by what the custom compiler’s OTP-28 toolchain can build: projects whose dependencies do not compile under it never enter the dataset, so the results generalise to typecheckable Elixir code rather than to the language’s full ecosystem. The supervised dataset, while checker-verified, is modest in size (7498 both-pass entries) and resolves no cross-project dependency types at training time, so the model never observes the remote types the translator was forced to widen to `dynamic()`, precisely the cases where prediction would most help. Where the model does fail, the failures concentrate in framework code with non-obvious internal representations (packed-binary math types, opaque struct APIs) and in the recurring atom/integer-versus-binary confusion, suggesting that dependency-aware context at training time is the most promising lever for the next iteration.

Manual project preparation. Constructing and evaluating the corpus was not a fully automated pipeline: a number of projects required manual intervention before they would compile under the custom compiler. The development-branch Elixir is built against a specific Erlang/OTP release (OTP-28), so projects pinning an older toolchain, or depending on libraries incompatible with it, did not build out of the box; bringing them up required pinning compatible dependency versions and adjusting build settings. For instance, a project that aliased compilation to `-warnings-as-errors` turned the typechecker’s own warnings into hard failures and had to be relaxed. These interventions were applied uniformly and identically to both tracks, so they do not bias the comparison; but they mean the corpus cannot yet be regenerated end-to-end without human effort, and that scaling to a substantially larger project set would demand either more robust automated environment resolution or an accepted loss of coverage.

A compiler under active development. The set-theoretic type system targeted in this work ships in a development-branch compiler (Elixir 1.21.0-dev) whose type-checking behaviour is not yet stable, and the translation pipeline depends directly on it. This coupling produced concrete incompatibilities between the translated annotations and the compiler. The clearest is an arity mismatch in the `@assert_type` form assertions the translation emits: for a function declaring specifications at several arities alongside a default-argument clause, the assertion generated for a higher arity could be associated by the compiler with a lower-arity clause, aborting compilation of the whole file; resolving these required repositioning the offending `@specs` by hand. Such mismatches are a property of a young, moving target rather than of the translation itself, so the empirical results reported here are tied to a particular snapshot of the compiler and can be expected to shift, most plausibly to improve, as its type checker and its handling of assertions mature.

Appendix. Semantics of Field Types in TypeSpecs

To understand the field type translation, the comprehension of field type notation has to precede. Field types are divided into mandatory types and non-mandatory types that could be denoted by `required` and `optional` in their key-types as the field type translation in Figure 1.9 suggests. Although it is natural to discern whether a pair is required or optional by the denotation, the semantics does not follow that intuition. By default when type-checked with Dialyzer, literal types are mandatory even with `optional` denotation. On the other hand, non-literal types are always optional whether they are denoted by `required` or not.

Semantics in Translation of Field Types

The only case when denotation has practical effect on field types is when used with literal types such as singleton atoms, digits, interval, and tuples. More specifically, `optional` denotation changes them to be optional:

```
1 @spec fun(%{:ok => binary()}) :: binary()  
2 @spec fun(%{optional(:ok) => binary()}) :: binary()
```

Line 1 specifies that the input map type has to contain a field that maps the key value `:ok` to a binary value. Whereas the same field is not mandatory with line 2, but when a field with the key `:ok` exists, its value has to be of binary type, which logic applies to all the other optional field types.

```
3 @spec fun(%{required(atom()) => binary()}) :: binary()  
4 @spec fun(%{optional(atom()) => binary()}) :: binary()
```

Even though denoted by `required` in line 3, it works the same as line 4 with `optional` because `atom()` is a non-literal type.

Essentially, these are the key semantics for translating TypeSpecs to Elixir Types in field types, that is, only the key types as singleton atom types are taken as the mandatory field, hence translated without `if_set` on value types.

Semantics in Merge of Field Types

As the merge function is merging a field type to an accumulated list of field types, every merge starts with the left-most field type to an empty list, which clearly results in a list of the merging field type itself. Although the intro is simple and obvious, the rest of Figure 1.12 lists the possible results and conditions of merges that depend on accumulated field types and merging field type. Therefore, the purpose of following examples is to demonstrate Figure 1.12 and show the steps we take to approximate the *undefined* key types using promotion.

Note that in each Condition Check of the figures below, we only use the form $K \in [K' \Rightarrow T]$ for checking if the accumulated list contains a promoted key type from a merging field type. This reduces the verbosity of the formal expression and reflects the actual implementation of searching.

An approximated key type found in the list of precedent fields

No subtype relationship

Case 1: Rule (M2) To merge a disjoint field type is only to append a merging field type to the accumulated list.

```
5 @spec fun(%{:a() => binary(), 1..10 => integer()}) :: binary()
```

Map Type = $\% \{a \Rightarrow \text{binary}(), 1..10 \Rightarrow \text{integer}()\}$
 Promoted Form = $\% \{a^a \Rightarrow \text{binary}(), \text{integer}()^{1..10} \Rightarrow \text{if_set}(\text{integer}())\}$
 First Merge $\rightsquigarrow [a^a \Rightarrow \text{binary}()] \oplus [\text{integer}()^{1..10} \Rightarrow \text{if_set}(\text{integer}())]$
 Subtype Check : $\text{integer}() \notin [a^a \Rightarrow \text{if_set}(\text{binary}())]$
 Second Merge $\rightsquigarrow [a^a \Rightarrow \text{binary}(), \text{integer}()^{1..10} \Rightarrow \text{if_set}(\text{integer}())]$

Figure 3.1: Merge for Line 5

Case 2: Rule (M3) The following case shows that the original key type of the first field is a super type of the original key type of the second field, i.e., $\text{atom}() \leq \text{atom}()$:

```
6 @spec fun(%{atom() => binary(), atom() => integer()}) :: binary()
```

Although merge is available without approximation, we show how the two fields are merged with the conservative merge with approximation:

Map Type = $\% \{\text{atom}() \Rightarrow \text{binary}(), \text{atom}() \Rightarrow \text{integer}()\}$
 Promoted Form = $\% \{\text{atom}()^{\text{atom}()'} \Rightarrow \text{binary}(), \text{atom}()^{\text{atom}()} \Rightarrow \text{integer}()\}$
 First Merge $\rightsquigarrow [\text{atom}()^{\text{atom}()'} \Rightarrow \text{binary}()] \oplus [\text{atom}()^{\text{atom}()} \Rightarrow \text{integer}()]$
 Condition Check : $\text{atom}() \in [\text{atom}()^{\text{atom}()'} \Rightarrow \text{binary}()]$
 $\wedge (\text{atom}() \cap \text{atom}()) \leq (\text{atom}() \cap \text{atom}())'$
 Second Merge $\rightsquigarrow [\text{atom}()^{\text{atom}()'} \Rightarrow \text{binary}()]$

The superscripted $\text{atom}()'$ is the same as $\text{atom}()$; we append a prime in accordance with our merge definition to display that it is from the list.

Figure 3.2: Merge for Line 6

Since the subtype relation is satisfied between the two key types, the former field completely overwrite the latter.

Case 3: Rule (M4) Despite the clearance, the Case 2 does not show the necessity of promotion. In this case, we demonstrate its practicality with literal types that must be approximated to key types in Elixir Types:

```
7 @spec fun(%{1..10 => binary(), 6..15 => integer()}) :: binary()
```

By mere approximation, both key types are to be $\text{integer}()$, and that trivially satisfies the condition of Rule (M3) which rule ignores the merging field. However, that leads to the wrong translation since the original types do not meet the subtype relationship. Therefore, when found the approximated type in the precedent list, we check the subtype relationship by involving the original types.

Map Type = $\% \{1..10 \Rightarrow \text{binary}(), 6..15 \Rightarrow \text{integer}()\}$
 Promoted Form = $\% \{\text{integer}()^{1..10} \Rightarrow \text{if_set}(\text{binary}()), \text{integer}()^{6..15} \Rightarrow \text{if_set}(\text{integer}())\}$
 First Merge $\rightsquigarrow [\text{integer}()^{1..10} \Rightarrow \text{if_set}(\text{binary}())] \oplus [\text{integer}()^{6..15} \Rightarrow \text{if_set}(\text{integer}())]$
 Condition Check : $\text{integer}() \in [\text{integer}()^{1..10} \Rightarrow \text{if_set}(\text{binary}())]$
 $\wedge (\text{integer}() \cap 6..15) \not\subseteq (\text{integer}() \cap 1..10)$
 Second Merge $\rightsquigarrow [\text{integer}()^{1..15} \Rightarrow \text{if_set}(\text{integer}() \text{ or } \text{binary}())]$

The superscripted interval types in Second Merge are unified as a single interval type since their ranges overlap.

Figure 3.3: Merge for Line 7

By checking the relationship between the key types, we unify the original key types as well as their value types so that the merge function does not make loss of information.

Merging singleton atom to the list

Case 4: Rule (M5) Even though the logic of this case is the same as the Case 2, we make it explicit on the ground of $:k \neq \text{atom}()$.

```
8 @spec fun(%{atom() => binary(), :a => integer()}) :: binary()
```

This is a specific case when a field type with `atom()` as its key type comes before the one with a singleton atom key type. Apparently, the former overwrites the latter regardless the latter either optional or required.

Map Type = $\% \{\text{atom}() \Rightarrow \text{binary}(), :a \Rightarrow \text{integer}()\}$
 Promoted Form = $\% \{\text{atom}()^{\text{atom}()} \Rightarrow \text{if_set}(\text{binary}()), :a^a \Rightarrow \text{integer}()\}$
 First Merge $\rightsquigarrow [\text{atom}()^{\text{atom}()} \Rightarrow \text{if_set}(\text{binary}())] \oplus [:a^a \Rightarrow \text{integer}()]$
 Condition Check : $\text{atom}() \in [\text{atom}()^{\text{atom}()} \Rightarrow \text{if_set}(\text{binary}())]$
 Second Merge $\rightsquigarrow [\text{atom}()^{\text{atom}()} \Rightarrow \text{binary}()]$

Figure 3.4: Merge for Line 8

Case 5: Rule (M6) Although marked as an optional field type, a field type with its key type as a singleton atom type can be shifted to a required field type. That is the case when a required singleton atom key type comes after a field type with the same key type that is optional.

```
9 @spec fun(%{optional(:a) => binary(), :a => integer()}) :: binary()
```

Hence, the first field type becomes mandatory while the second field type is overwritten:

```

Map Type    = % { optional (:a) => binary(), :a => integer() }
Promoted Form = % { :aa' => if_set(binary()), :aa => integer() }
First Merge  ~> [ :aa' => if_set(binary()) ] ⊕ [ :aa => integer() ]
Condition Check : :a ∈ [ :aa' => if_set(binary()) ]
Second Merge ~> [ :aa' => binary() ]

```

The superscripted $:a'$ is the same as $:a$; we append a prime in accordance with our merge definition to display that it is from the list.

Figure 3.5: Merge for Line 9

Appendix. Dialyzer-Only Warning Examples

This appendix gives representative Dialyzer-only entries—functions on which Dialyzer raises a warning but the new typechecker raises none—one from each of the three groups of Subsection 2.3.4: a whole-program category (`no_return`), a locally-checkable category (`pattern_match`), and an opacity category (`opaque_match`). Each block shows the function definition and the Dialyzer message (type dumps elided); the reason the typechecker does not reproduce a given category is established in Subsection 2.3.4.

no_return (whole-program). On `Mobilizon.Web.WebFingerController.host_meta/2`:

```
1 def host_meta(conn, _params) do
2   xml = WebFinger.host_meta()
3   conn |> put_resp_content_type("application/xrd+xml") |> send_resp(200, xml)
4 end
```

Dialyzer warns:

```
1 Function host_meta/2 has no local return.
```

pattern_match (locally checkable). On `Mobilizon.Service.Activity.Utils.maybe_inserted_at/0`:

```
1 def maybe_inserted_at do
2   if @env == :test do
3     %{}
4   else
5     %{"inserted_at" => DateTime.utc_now()}
6   end
7 end
```

Dialyzer warns:

```
1 Legacy warning: The test 'dev' == 'test' can never evaluate to 'true'
```

Dialyzer constant-folds the module attribute `@env` (`:dev` at compile time) and finds the `:test` branch dead; the module-local typechecker does not constant-fold it, so it leaves the branch reachable.

opaque_match (opacity). On `Livebook.Teams.WebSocket.send/4`:

```
1 def send(conn, websocket, ref, frame) when is_frame(frame) do
2   with {:ok, websocket, data} <- Mint.WebSocket.encode(websocket, frame),
3        {:ok, conn} <- Mint.WebSocket.stream_request_body(conn, ref, data) do
4     {:ok, conn, websocket}
5   else
6     {:error, %Mint.HTTP1{} = conn, exception} when is_exception(exception) ->
7       {:error, conn, websocket, Exception.message(exception)}
8   end
9 end
```

Dialyzer warns:

```
1 Attempted to pattern match against the internal structure of an opaque term.
2 Type:
3 Mint.HTTP.t() | Mint.WebSocket.t()
4 Pattern:
5 %Mint.HTTP1{}
6 This breaks the opaqueness of Mint.HTTP.t() | Mint.WebSocket.t().
```

The new type system does not model `@opaque` types, so it has no opacity boundary to violate.

Appendix. Both-Warned Examples

This appendix complements the previous one with entries from the 199-entry *both-warned* set (Subsection 2.3.6): definitions faulty enough that Dialyzer and the new typechecker independently reject them. The two blocks illustrate the two ways the typechecker co-rejects—for a locally-checkable category it reproduces Dialyzer’s finding directly, and for a whole-program or opacity category it cannot, so it rejects instead on a coexisting body-level fault. Each block gives the function definition, the Dialyzer message, and the typechecker’s finding (bodies and type dumps elided).

Direct reproduction (call). On `Pleroma.HTTP.get/3`:

```
1 def get(url, headers \\ [], options \\ [])
2   def get(nil, _, _) do
3     nil
4   end
5   def get(url, headers, options) do
6     request(:get, url, nil, headers, options)
7   end
```

Dialyzer:

```
1 The function call will not succeed.
2 Pleroma.HTTP.request(:get, _url :: any(), nil, _headers :: any(), _options :: any())
3 breaks the contract
4 (method(), Request.url(), String.t(), Request.headers(), :elixir.keyword()) ::
5 ...
```

Typechecker:

```
1 incompatible types given to request/5
```

The finding is a call-site argument mismatch, which the typechecker checks body-locally and reproduces directly.

Co-rejection on a separate fault (call_without_opaque). On `ExCubicIngestion.ProcessIngestion.error/1`:

```
1 def error(load_rec) do
2   Ecto.Multi.new()
3   |> Ecto.Multi.update(:update_status, CubicLoad.change(load_rec, %{status: "erroring"}))
4   |> Oban.insert(:error_job, Error.new(%{load_rec_id: load_rec.id}))
5   |> Repo.transaction()
6 end
```

Dialyzer:

```
1 Legacy warning: The call 'Elixir.Ecto.Multi':update("#{__struct__}':'Elixir.Ecto.Multi',
  ...
```

Typechecker:

```
1 asserted return type does not match the function body
```

The opacity finding is outside the typechecker’s model, so it cannot reproduce it; it co-rejects the definition on a coexisting return-type mismatch instead.

Appendix. Typechecker-Only Warning Examples

Of the 3636 entries the typechecker rejects but Dialyzer accepts, 691 are return-type findings; the larger unreachable-clause, argument-mismatch, and map-key groups are treated in Subsection 2.3.5. The return-type findings split into 390 whose annotation carries a `dynamic()` (no static verdict after root-hoisting), 125 *cannot-verify* cases (the body’s return flows from an intra-project or un-spec’d call that module-local inference assumes is `dynamic()`), and 176 purely-static disagreements. The purely-static group is where a true static verdict is possible: a genuine leak is a body that provably returns a value outside its declared type, which Dialyzer’s optimism accepts. This appendix gives representative genuine leaks and one worked example of each recurring artefact category; it is illustrative rather than exhaustive.

Representative genuine return-type leaks

Each returns a value the declared type excludes—typically a clause that leaks `nil` or an error tuple, or a spec that under-declares the returned atoms.

Function	Body returns	Declared
Scenic.PubSub.publish/2	<code>:ok</code> or <code>{:error, :not_registered}</code>	<code>:ok</code>
Ash.UUIDv7.generate/0	<code>:error</code> or <code>binary()</code>	<code>binary()</code>
Timex.day_name/1	<code>nil</code> or <code>binary()</code>	<code>binary()</code> or <code>{:error, :invalid_weekday_number}</code>
Cldr.Calendar.Coptic.quarter_of_year/3	<code>{:error, :not_defined}</code>	<code>integer()</code>
Livebook.Teams.encrypt/2	<code>nil</code> or <code>binary()</code>	<code>binary()</code>

Artefact categories

Not every return disagreement is a genuine leak; the two recurring non-genuine kinds are the conservatism of module-local inference and its over-broadening.

Cannot verify. `Ash.DataLayer.rollback/3`.

```
1 def rollback(data_layer_module, resource, term) do
2   apply(data_layer_module, :rollback, [resource, term])
3 end
```

Inferred `dynamic()`; declared `none()`. The body returns the result of a dynamic `apply/3` that inference cannot resolve, so the checker cannot establish the declared type, whereas Dialyzer’s whole-program success typing resolves the callee.

Over-broadened inference. `Scenic.Color.to_ga/1`.

```
1 def to_ga(g) when is_uint8(g) do
2   {@ga, {g, 255}}
3 end
4 def to_ga({g, a}) when is_uint8(g) and is_uint8(a) do
5   {@ga, {g, a}}
6 end
7 def to_ga({r, g, b}) when is_uint8(r) and is_uint8(g) and is_uint8(b) do
8   {@ga, {do_rgb_to_g({r, g, b}), 255}}
9   # ...
```

Inferred `{:color_ga, integer() or {integer(), integer()}}`; declared `{:color_ga, {integer(), integer()}}`. Inference adds a spurious union arm—a bare integer—that the body never returns.

Appendix. Source Projects

The dataset is constructed from the following 383 public repositories, each pinned to the exact commit used during construction and available at the listed location.

Table 3.3: Source repositories and pinned commits used for dataset construction.

Repository	Commit
akkoma.dev/AkkomaGang/akkoma	180433759
framagit.org/kaihuri/mobilizon	029ad665a
github.com/absinthe-graphql/absinthe	8f9816a6
github.com/acalejos/curl_req	e4705e3
github.com/acalejos/exgboost	782d00f
github.com/acalejos/instructor_ex	60f3364
github.com/acalejos/liteskill-oss	bc1296f
github.com/acalejos/truly	331193b
github.com/acalejos/zigler	6837801
github.com/am-kantox/ack	9d2c93e
github.com/am-kantox/agency	a77dc3f
github.com/am-kantox/antenna	099f437
github.com/am-kantox/apic	e13eb82
github.com/am-kantox/bex	6bf33c6
github.com/am-kantox/camarero	0f88d22
github.com/am-kantox/cloister	687956d
github.com/am-kantox/distributed_supervisor	1d44bbb
github.com/am-kantox/elixir-iteraptor	f9df686
github.com/am-kantox/enfiladex	1c793c4
github.com/am-kantox/envio	db8c89e
github.com/am-kantox/estructura	f90e597
github.com/am-kantox/ex_panda	44a4c63
github.com/am-kantox/finitomata	8c86f61
github.com/am-kantox/formulae	ca735ca
github.com/am-kantox/md	74a4c56
github.com/am-kantox/nasty	f13772b
github.com/am-kantox/nimble_options_ex	fc0c733
github.com/am-kantox/peeper	2119d92
github.com/am-kantox/persistomata	d0aff42
github.com/am-kantox/ragex_yeesh	8f8fbed
github.com/am-kantox/rambla	8373be6
github.com/am-kantox/siblings	78ee09a
github.com/am-kantox/solo	50d4461
github.com/am-kantox/tarearbol	f2b9c42
github.com/am-kantox/telemetry	3386efc
github.com/am-kantox/tempus	5660312
github.com/am-kantox/tempus_sql	70882a5
github.com/am-kantox/tic_tac_toe_checker	0feeac6
github.com/am-kantox/vela	e946664
github.com/ash-project/ash	6c0357d97
github.com/ash-project/ash_ai	3c14454
github.com/ash-project/ash_authentication_oauth2_server	2681877
github.com/ash-project/ash_cloak	51c19a8

Table 3.3 continued

Repository	Commit
github.com/ash-project/ash_graphql	c408083
github.com/ash-project/ash_hq	d90e367
github.com/ash-project/ash_json_api	603c5f6
github.com/ash-project/ash_json_api_wrapper	83f8e60
github.com/ash-project/ash_lua	eea4c60
github.com/ash-project/ash_money	cb9660d
github.com/ash-project/ash_oban	1445b47
github.com/ash-project/ash_ops	0e3f96e
github.com/ash-project/ash_paper_trail	228ddce
github.com/ash-project/ash_phoenix	0d0421c
github.com/ash-project/ash_postgres	d059a449
github.com/ash-project/ash_rate_limiter	a976db8
github.com/ash-project/ash_scim	4e059fa
github.com/ash-project/ash_slug	4ba9e47
github.com/ash-project/ash_state_machine	77d7379
github.com/ash-project/ash_storage	4c32e6e
github.com/ash-project/ash_sync	9aef63d
github.com/ash-project/ash_typescript	e179d96
github.com/ash-project/crux	3c8028c
github.com/ash-project/evals	3c3b231
github.com/ash-project/igniter	5a167c0
github.com/ash-project/iterex	d649b91
github.com/ash-project/reactor	c11578c
github.com/ash-project/reactor_process	d5bc8f5
github.com/ash-project/reactor_req	3b3d9af
github.com/ash-project/simple_sat	22ace41
github.com/ash-project/spark	51fa24d
github.com/beam-telemetry/telemetry_metrics	254e6dc
github.com/beam-telemetry/telemetry_metrics_prometheus	189d0cd
github.com/beam-telemetry/telemetry_metrics_prometheus_core	fe82cc5
github.com/beam-telemetry/telemetry_metrics_statsd	a666924
github.com/bencheeorg/benchee	55a5e60
github.com/bencheeorg/statistex	6695de6
github.com/bitwalker/artificery	afbfd57
github.com/bitwalker/timex	5ad1b82
github.com/bitwalker/toml-elixir	e32c899
github.com/bitwalker/uniq	48ce1ef
github.com/codedge-llc/pigeon	712d5c2
github.com/commanded/commanded	936dbb1
github.com/commanded/eventstore	930f921
github.com/danielberkompas/elasticsearch-elixir	fba3359
github.com/danielberkompas/ex_twilio	8378ee5
github.com/danielberkompas/ex_twiml	982fd65
github.com/danielberkompas/immortal	416c675
github.com/danielberkompas/number	67d5165
github.com/dashbitco/broadway	d3a668c
github.com/dashbitco/broadway_dashboard	8df6b5b
github.com/dashbitco/broadway_kafka	309b5a8

Table 3.3 continued

Repository	Commit
github.com/dashbitco/lazy_html	e65f0cd
github.com/dashbitco/mox	53f7908
github.com/dashbitco/nimble_options	f16af25
github.com/dashbitco/nimble_pool	ca7e7bd
github.com/dashbitco/nimble_totp	ddc79fe
github.com/edgurgel/ham	6edecb5
github.com/edgurgel/httparrot	3901fe2
github.com/edgurgel/httpoison	18be3fe
github.com/edgurgel/logflare_api_client	b066f10
github.com/edgurgel/postgrex	412b555
github.com/edgurgel/solid	1084861
github.com/elixir-cldr/cldr	b8c28ed0b
github.com/elixir-cldr/cldr-2.43.0-dev	5983abf
github.com/elixir-cldr/cldr47	0b46517
github.com/elixir-cldr/cldr48	8de4b8e
github.com/elixir-cldr/cldr_calendars	cb30e78
github.com/elixir-cldr/cldr_calendars_composite	3828fb5
github.com/elixir-cldr/cldr_calendars_coptic	b8ca399
github.com/elixir-cldr/cldr_calendars_ethiopic	687426a
github.com/elixir-cldr/cldr_calendars_format	f4ca07e
github.com/elixir-cldr/cldr_calendars_lunisolar	2eb2c81
github.com/elixir-cldr/cldr_calendars_persian	8a50a46
github.com/elixir-cldr/cldr_currencies	b4bd855
github.com/elixir-cldr/cldr_dates_times	83f5ce0
github.com/elixir-cldr/cldr_html	cd39399
github.com/elixir-cldr/cldr_lists	72141f6
github.com/elixir-cldr/cldr_locale_display	f8452be
github.com/elixir-cldr/cldr_messages	f457629
github.com/elixir-cldr/cldr_numbers	f07e892
github.com/elixir-cldr/cldr_person_names	7be4214
github.com/elixir-cldr/cldr_plugs	760432e
github.com/elixir-cldr/cldr_routes	35c89d6
github.com/elixir-cldr/cldr_territories	90efd2f
github.com/elixir-cldr/cldr_trans	332061a
github.com/elixir-cldr/cldr_units	0a87662
github.com/elixir-cldr/cldr_units_sql	a2be771
github.com/elixir-cldr/cldr_utils	3ec7ca9
github.com/elixir-cldr/magical	78fd703
github.com/elixir-ecto/connection	e17de7e
github.com/elixir-ecto/db_connection	4d7d6d8
github.com/elixir-ecto/ecto	9fee9456
github.com/elixir-ecto/ecto_sql	91c6f8a
github.com/elixir-ecto/excoding	abb97f7
github.com/elixir-ecto/myxql	4202c44
github.com/elixir-ecto/postgrex	03717e9
github.com/elixir-ecto/table	5704327
github.com/elixir-ecto/tds	61d2f27
github.com/elixir-explorer/explorer	f82e05d

Table 3.3 continued

Repository	Commit
github.com/elixir-explorer/fss	71a1f71
github.com/elixir-grpc/grpc-reflection	1eb9d97
github.com/elixir-lang/elixir	28ffbcfe8
github.com/elixir-lang/ex_doc	25523c9d
github.com/elixir-lang/gen_stage	72198f8
github.com/elixir-lang/logger_backends	947b72e
github.com/elixir-lang/registry	51d651a
github.com/elixir-lsp/elixir_ls	d1bfbbcc
github.com/elixir-lsp/elixir_sense	3ec7dba1
github.com/elixir-lsp/erl2ex	04f93e5
github.com/elixir-mint/castore	d7c27ff
github.com/elixir-mint/hpax	7eea90a
github.com/elixir-mint/mint	99ebaca
github.com/elixir-mint/mint_web_socket	0c630f75
github.com/elixir-nx/bumblebee	0fd8114
github.com/elixir-nx/complex	ab2149e
github.com/elixir-nx/emlx	41b2e83
github.com/elixir-nx/nx	1c3d86eb
github.com/elixir-nx/safetensors	29df313
github.com/elixir-nx/scidata	ea7a488
github.com/elixir-nx/tokenizers	d3081c4
github.com/elixir-nx/xla	90e1eed
github.com/elixir-plug/mime	354c426
github.com/elixir-plug/plug	2463704
github.com/elixir-plug/plug_cowboy	74dab49
github.com/elixir-plug/plug_crypto	cdbd36c
github.com/elixir-wallaby/wallaby	13e1394
github.com/elixir-wallaby/web_driver_client	1eeaf7a
github.com/ericmj/awardflights	4c1f0eb
github.com/ericmj/decimal	eb75d13
github.com/ex-aws/ex_aws	06f3d50
github.com/HashNuke/hound	1be61d4
github.com/HashNuke/ircxd	b063343
github.com/hexpm/bob	550e071
github.com/hexpm/hex	0eb1763
github.com/hexpm/hex_solver	288dfef
github.com/hexpm/hexdocs	a7eeffe
github.com/hexpm/hexpm	5e32ce30
github.com/jeremyjh/dialyxir	3553678
github.com/joken-elixir/joken	01fcd98
github.com/josevalim/bandit	53fdf06
github.com/josevalim/codepagex	e1b6e6f
github.com/josevalim/erlex	0548765
github.com/josevalim/finch	047a417
github.com/josevalim/fresh	d819990
github.com/josevalim/igniter	94aadda
github.com/josevalim/spitfire	c3f874e
github.com/josevalim/surface	5b9f0b7

Table 3.3 continued

Repository	Commit
github.com/josevalim/website-1	78db368
github.com/lexical-lsp/lexical	477e8b4
github.com/livebook-dev/adbc	9f429a8
github.com/livebook-dev/elixirkit	77f0e34
github.com/livebook-dev/kino	9585541
github.com/livebook-dev/kino_bumblebee	8d40d77
github.com/livebook-dev/kino_explorer	618f447
github.com/livebook-dev/kino_maplibre	92bac7d
github.com/livebook-dev/kino_pythonx	52a4937
github.com/livebook-dev/kino_vega_lite	3aded40
github.com/livebook-dev/livebook	f6ade138
github.com/livebook-dev/maplibre	460d77c
github.com/livebook-dev/pythonx	1700d8b
github.com/livebook-dev/req_athena	57f8790
github.com/livebook-dev/req_bigquery	9a7fb92
github.com/livebook-dev/req_ch	d26562f
github.com/livebook-dev/vega_lite	5dd5dd5
github.com/livebook-dev/vega_lite_convert	6e0737e
github.com/mbta/alerts_concierge	179464e5
github.com/mbta/alerts_viewer	58c4580
github.com/mbta/api	c502179
github.com/mbta/arrow	7b33342
github.com/mbta/bamboo	6ae35de
github.com/mbta/broadway_kinesis	5cebef2
github.com/mbta/commuter_rail_boarding	b7cae9b
github.com/mbta/concentrate	25ca4e0
github.com/mbta/data_platform	ef7c69c
github.com/mbta/delta	2f11f91
github.com/mbta/diskusage_logger	f421101
github.com/mbta/document_viewer	f48e4ba
github.com/mbta/dotcom	ce2f50bad
github.com/mbta/gtfs_realtime_viz	f83212b
github.com/mbta/http_stage	fea6d81
github.com/mbta/ja_serializer	efb1d44
github.com/mbta/logger_splunk_backend	68eeca4
github.com/mbta/mbta_v3_api	081d85f
github.com/mbta/mobile_app_backend	2ddbb48
github.com/mbta/open_trip_planner_client	8658680
github.com/mbta/openid_connect	ddafedc
github.com/mbta/orbit	ded5d2b
github.com/mbta/phil	1d7f93e
github.com/mbta/phoenix_swagger	c22a2f3
github.com/mbta/prediction_analyzer	9137df7
github.com/mbta/react-phoenix	d95a346
github.com/mbta/realtime_signs	eb43d48
github.com/mbta/ride_along	7c49cad
github.com/mbta/screen_checker	b5dc0b1
github.com/mbta/screens	0f7a34a1

Table 3.3 continued

Repository	Commit
github.com/mbta/screens-config-lib	9157461
github.com/mbta/sentry-elixir	ce417a6
github.com/mbta/solverl	3d972db
github.com/mbta/subwayside_ui	77ea0ff
github.com/mbta/tablespoon	c8edec8
github.com/mbta/transit_data_reports	ad1f092
github.com/mbta/trike	10bc45c
github.com/mbta/uncooked-gps	3393a24
github.com/membraneframework/beamchmark	7856e22
github.com/membraneframework/boombox	119210c
github.com/membraneframework/bunch	e3ff1c3
github.com/membraneframework/bundlex	fd4dbe7
github.com/membraneframework/ex_hls	f610595
github.com/membraneframework/ex_kamailio	eda2d98
github.com/membraneframework/ex_libnice	f009103
github.com/membraneframework/ex_libsrt	511c79d
github.com/membraneframework/ex_libsrtp	2d724e8
github.com/membraneframework/ex_m3u8	afe6b58
github.com/membraneframework/ex_sdp	c1fe285
github.com/membraneframework/kino_membrane	bab5d6a
github.com/membraneframework/membrane_aac_fdk_plugin	2c71376
github.com/membraneframework/membrane_aac_format	aaa1561
github.com/membraneframework/membrane_aac_plugin	e985b97
github.com/membraneframework/membrane_audio_mix_plugin	adfcfb5
github.com/membraneframework/membrane_audiometer_plugin	0a1380b
github.com/membraneframework/membrane_av1_format	e474b3e
github.com/membraneframework/membrane_av1_plugin	b446be6
github.com/membraneframework/membrane_camera_capture_plugin	31166b8
github.com/membraneframework/membrane_core	016ca6e5
github.com/membraneframework/membrane_element_gcloud_speech_to_text	a8cd950
github.com/membraneframework/membrane_ertmp_plugin	9458d52
github.com/membraneframework/membrane_ffmpeg_video_filter_plugin	fa251bc
github.com/membraneframework/membrane_file_plugin	da7f139
github.com/membraneframework/membrane_flac_plugin	fb8a2b8
github.com/membraneframework/membrane_flv_plugin	9fc1be9
github.com/membraneframework/membrane_g711_plugin	0a5c3b7
github.com/membraneframework/membrane_gemini_demo	963c2e2
github.com/membraneframework/membrane_gemini_plugin	5982cdb
github.com/membraneframework/membrane_h264_ffmpeg_plugin	b147d41
github.com/membraneframework/membrane_h26x_plugin	f469361
github.com/membraneframework/membrane_http_adaptive_stream_plugin	9d8c6c5
github.com/membraneframework/membrane_ivf_plugin	89548e5
github.com/membraneframework/membrane_matroska_plugin	02822c1
github.com/membraneframework/membrane_mp3_lame_plugin	2bb12b0
github.com/membraneframework/membrane_mp3_mad_plugin	98b4d56
github.com/membraneframework/membrane_mp4_plugin	2b3b89b
github.com/membraneframework/membrane_mpegaudio_format	aee638a
github.com/membraneframework/membrane_mpegts_plugin	0ddafda

Table 3.3 continued

Repository	Commit
github.com/membraneframework/membrane_ogg_plugin	7d9dc9a
github.com/membraneframework/membrane_opus_plugin	adf5a40
github.com/membraneframework/membrane_overlay_plugin	fa1a820
github.com/membraneframework/membrane_portaudio_plugin	2f49427
github.com/membraneframework/membrane_precompiled_dependency_provider	068aa7e
github.com/membraneframework/membrane_raw_audio_format	38f1aee
github.com/membraneframework/membrane_raw_audio_parser_plugin	71c3e90
github.com/membraneframework/membrane_raw_video_format	72e6d88
github.com/membraneframework/membrane_realtimer_plugin	3591f33
github.com/membraneframework/membrane_rpcam_plugin	8e3b4b0
github.com/membraneframework/membrane_rtmp_plugin	86579f4
github.com/membraneframework/membrane_rtp_aac_plugin	8bebe25
github.com/membraneframework/membrane_rtp_format	651dd42
github.com/membraneframework/membrane_rtp_g711_plugin	3f01dde
github.com/membraneframework/membrane_rtp_h264_plugin	138516b
github.com/membraneframework/membrane_rtp_opus_plugin	9631fbd
github.com/membraneframework/membrane_rtp_plugin	8198ea8
github.com/membraneframework/membrane_rtp_vp8_plugin	3796ace
github.com/membraneframework/membrane_rtsp	beec826
github.com/membraneframework/membrane_scissors_plugin	2c7ba8d
github.com/membraneframework/membrane_sdk	45ee054
github.com/membraneframework/membrane_smelter_plugin	5080fff
github.com/membraneframework/membrane_stream_plugin	ab8c4fa
github.com/membraneframework/membrane_tcp_plugin	c9760d4
github.com/membraneframework/membrane_telemetry_metrics	81e7df5
github.com/membraneframework/membrane_timestamp_queue	1144930
github.com/membraneframework/membrane_transcoder_plugin	b61945e
github.com/membraneframework/membrane_udp_plugin	efb7500
github.com/membraneframework/membrane_video_merger_plugin	531457d
github.com/membraneframework/membrane_vk_video_plugin	dadd7ab
github.com/membraneframework/membrane_vpx_plugin	0c8ac36
github.com/membraneframework/membrane_wav_plugin	e5731ff
github.com/membraneframework/membrane_webrtc_plugin	26424a3
github.com/membraneframework/membrane_whisper_plugin	cb87682
github.com/membraneframework/membrane_yolo_plugin	174d91d
github.com/membraneframework/shmex	60a07db
github.com/membraneframework/unifex	16ce290
github.com/michalmuskala/jason	4ede428
github.com/mortont/axon_onnx	a9256dd
github.com/mtrudel/hap	3110913
github.com/mtrudel/hpax	add8029
github.com/mtrudel/machete	ff6d1df
github.com/mtrudel/thousand_island	cbc55d1
github.com/nerves-project/alarmist	75bb027
github.com/nerves-project/nerves	6ef898a
github.com/nerves-project/nerves_bootstrap	cc590b4
github.com/nerves-project/nerves_burner	ca25c2b
github.com/nerves-project/nerves_examples	13dad4e

Table 3.3 continued

Repository	Commit
github.com/nerves-project/nerves_leds	9b3e989
github.com/nerves-project/nerves_logging	91ce5be
github.com/nerves-project/nerves_motd	b25ce0d
github.com/nerves-project/nerves_runtime	b62aeb9
github.com/nerves-project/nerves_ssh	78aa44e
github.com/nerves-project/nerves_system_linter	e305b6c
github.com/nerves-project/nerves_uevent	f9b6f53
github.com/nerves-project/property_table	817c764
github.com/nerves-project/ramoops_logger	53122aa
github.com/nerves-project/ring_logger	a5602a7
github.com/nerves-project/shoehorn	c4324fd
github.com/nerves-project/ssh_subsystem_fwup	b16d623
github.com/nerves-project/uboot_env	a146bc0
github.com/oban-bg/oban	9dd5106
github.com/parrotty/extwitter	854b6a2
github.com/philss/floki	143df1f
github.com/philss/req_athena	c2bc1d4
github.com/philss/table_kitty	bb08826
github.com/phoenixframework/firenest	335032b
github.com/phoenixframework/phoenix	6ceade428
github.com/phoenixframework/phoenix_ecto	7225df6
github.com/phoenixframework/phoenix_html	b21a2d0
github.com/phoenixframework/phoenix_html_helpers	1fdbdd9
github.com/phoenixframework/phoenix_live_dashboard	4468d13
github.com/phoenixframework/phoenix_live_view	1fedd12d
github.com/phoenixframework/phoenix_pubsub	228b82e
github.com/phoenixframework/phoenix_template	d3bc0d3
github.com/phoenixframework/phoenix_view	87159b5
github.com/phoenixframework/plds	8f46106
github.com/phoenixframework/websock_adapter	e7bbe13
github.com/plausible/analytics	6be0b06194
github.com/pow-auth/assent	1707085
github.com/pow-auth/pow	fe09223
github.com/pow-auth/pow_assent	9833bc4
github.com/rrrene/credo	89457a16
github.com/rrrene/html_sanitize_ex	e54b2e7
github.com/ScenicFramework/font_metrics	cf08b4c
github.com/ScenicFramework/scenic	e0ae569
github.com/ScenicFramework/scenic_clock	54bdd24
github.com/sneako/ex_kpl	b4f99f3
github.com/sneako/nimble_pool	2444fd0
github.com/thechangelog/changelog.com	a17e5a9c
github.com/whatyouhide/nimble_lz4	e89a874
github.com/whatyouhide/redix	90ecf9f
github.com/whatyouhide/statix	d97e684
github.com/whatyouhide/stream_data	49958c0
github.com/whatyouhide/testing_aws_in_elixir	3452445
github.com/whatyouhide/xandra	d74c37b

Table 3.3 continued

Repository	Commit
github.com/whitfin/cachex	44ac7e4
github.com/wojtekmach/req	923aacf
github.com/woylie/flop	ba96436
gitlab.com/ssl-moon/ssl-moon	e32e26c

Bibliography

- Armstrong, J. (2007). *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf.
- Aronoff, J. and Dialyxir Contributors (2024). Dialyxir: Mix tasks to simplify use of Dialyzer in Elixir projects. <https://hexdocs.pm/dialyxir>. Accessed 2025.
- Castagna, G., Nguyễn, K., Xu, Z., Im, H., Lenglet, S., and Padovani, L. (2014). Polymorphic functions with set-theoretic types: Part 1: Syntax, semantics, and theory. In *Proceedings of the 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '14, pages 5–17, New York, NY, USA. ACM.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Duboc, G. (2024). *A Gradual Type System for Elixir*. PhD thesis, Université Paris Cité (IRIF). Supervised by Giuseppe Castagna.
- Elixir Core Team (2024a). alias, require, import, and use — Elixir documentation. <https://hexdocs.pm/elixir/alias-require-and-import.html>. Modules are represented by atoms; the alias String expands to the atom `"Elixir.String"`. Accessed 2025.
- Elixir Core Team (2024b). Gradual set-theoretic types — Elixir documentation. <https://hexdocs.pm/elixir/gradual-set-theoretic-types.html>. Accessed 2025.
- Elixir Core Team (2024c). Typespecs — Elixir documentation. <https://hexdocs.pm/elixir/typespecs.html>. Accessed 2025.
- Ericsson AB (2024a). Dialyzer — Erlang/OTP reference manual. <https://www.erlang.org/doc/man/dialyzer>. Accessed 2025.
- Ericsson AB (2024b). Erlang/OTP. <https://www.erlang.org>. Accessed 2025.
- Ericsson AB (2024c). Types and function specifications — Erlang reference manual. https://www.erlang.org/doc/reference_manual/typespec. Accessed 2025.
- Frisch, A., Castagna, G., and Benzaken, V. (2008). Semantic subtyping: Dealing set-theoretically with function, union, intersection, and negation types. *Journal of the ACM*, 55(4):19:1–19:64.
- Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y. K., Luo, F., Xiong, Y., and Liang, W. (2024). Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., and Lin, J. (2024). Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*.

- Lindahl, T. and Sagonas, K. (2006). Practical type inference based on success typings. In *Proceedings of the 8th ACM SIGPLAN Symposium on Principles and Practice of Declarative Programming*, PPDP '06, pages 167–178, New York, NY, USA. ACM.
- Lozhkov, A., Li, R., Ben Allal, L., Cassano, F., Lamy-Poirier, J., Tazi, N., et al. (2024). Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*.
- Mengesha, H. B. (2026). Assessing llm type prediction with multi-dimensional similarity metrics. Master's thesis, Ca'Foscari University of Venice. Supervised by Michele Bugliesi and Rocco Tripodi.
- Siek, J. G. and Taha, W. (2006). Gradual typing for functional languages. In *Proceedings of the 2006 Workshop on Scheme and Functional Programming*, pages 81–92.
- Valim, J. (2012). Elixir programming language. <https://elixir-lang.org>. Accessed 2025.
- Valim, J. (2026). Precision in type system design. <https://youtu.be/Ay-gnCqDw9o?is=IMHzHn4dvBMsFUpn>.
- Wang, Y., Le, H., Gotmare, A. D., Bui, N. D. Q., Li, J., and Hoi, S. C. H. (2023). Codet5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*.